

Известия за намаз **الأذان**

Камбанките на „Днес“ вече работят: в часа на молитвата идва известие с кратък звук или с езан, по желание и напомняне преди това. Всичко е локално – без интернет, без Firebase и без HMS, затова работи еднакво на телефони с Google услуги, на Huawei без тях и на iPhone.

expo-notifications

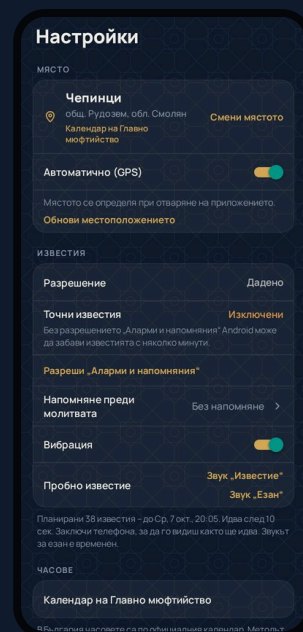
Фонова задача

Kotlin модул

GMS · HMS · iPhone

Jest – 112 теста

25 файла



Какво е решено в този етап

Само локални известия (AlarmManager на Android, UNUserNotificationCenter на iPhone) · до 62 известия напред, последното – „Отвори Езан“ · пренасрочване при всяка промяна, при отваряне и с фонова задача · напомняне 5–30 мин. преди · вибрация (Android) · „Аларми и напомняния“ за точни известия на Android 12+ · временни звуци до етап 6 · пълният езан на Android – етап 5.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нови пакети и нов build

03 Звуци и иконка

04 Native модул „AdhanNative“ (Kotlin, само Android)

05 Планът на известията

06 Службата: разрешение, канали, планиране

07 Екраните

08 Поправки след първата проба на телефона

09 Тестове

10 Проверка и край на етапа

Стъпка 1

Какво прави етапът

Един и същ път за всички известия: **настройки** → **план** → **телефонът**. Планът е чиста функция и е покрит с тестове; службата само го изпраща към телефона.

Дърво на проекта

камбанки („Днес“) + място + часове + напомняне + вибрация

planNotifications() src/domain/notifications.ts
(до 62 известия за 7 дни, по ред на часа)

rescheduleNotifications() src/services/notifications.ts
сравнява с вече планираните → трие само променените, добавя новите

- Android: AlarmManager (точен час с „Аларми и напомняния“)
- iPhone: UNUserNotificationCenter (до 64 чакащи)

Кога се пренасрочва: при стартиране • при връщане в приложението •
при промяна на настройка • фонова задача на ~6 часа • след рестарт (Android сам)

	ANDROID С GOOGLE (GMS)	HUAWEI БЕЗ GOOGLE (HMS)	IPHONE
Известия в часа	✓ AlarmManager	✓ AlarmManager – без Google	✓
Точен час	✓ с „Аларми и напомняния“	✓ с „Аларми и напомняния“	✓
Звук на алармата	до 28 сек. (временен)	до 28 сек. (временен)	до 30 сек. – ограничение на iOS
След рестарт	✓ пренасрочва сам	✓ пренасрочва сам	✓ пази ги системата
Фонова задача	WorkManager	WorkManager – без Google	BGTaskScheduler
Пълен езан, „Спри“, цял екран	етап 5	етап 5	етап 9 (AlarmKit)

Firebase (FCM) и HMS Push трябва само за известия, изпратени от сървър. Тук всичко се планира на самия телефон, затова не ги ползваме. Библиотеката носи код на Firebase, но той не се стартира без конфигурация – на Huawei не пречи.

Стъпка 2

Нови пакети и нов build

Трите пакета и native модулет имат native код, затова е нужен нов development build.

Инсталиране и нов build (Android)

```
npx expo install expo-notifications expo-background-task expo-task-manager  
Remove-Item -Recurse -Force android  
npx expo run:android
```

iPhone (изисква Apple Developer акаунт)

```
npx eas-cli@latest build --profile development --platform ios
```

app.json корен на проекта

- **expo-notifications** : бялата иконка за лентата с известия, златният цвят и двата звука – plugin-ът ги копира в **res/raw** (Android) и в приложението (iPhone).
- **expo-background-task** : на iPhone добавя **processing** и идентификатора на задачата в Info.plist.
- **android.permissions** : **SCHEDULE_EXACT_ALARM** – без него Android 12+ не планира точни известия. **POST_NOTIFICATIONS** и **RECEIVE_BOOT_COMPLETED** идват от самата библиотека.

app.json

98 реда

```
{  
  "expo": {  
    "name": "Adhan",  
    "slug": "adhan",  
    "version": "1.0.0",  
    "orientation": "portrait",  
    "icon": "./assets/icon.png",  
    "userInterfaceStyle": "dark",  
    "ios": {  
      "supportsTablet": false,  
      "bundleIdentifier": "com.ilkoadamov.adhan",  
      "infoPlist": {  
        "CFBundleAllowMixedLocalizations": true  
      }  
    },  
    "android": {  
      "adaptiveIcon": {  
        "backgroundColor": "#0F1B2D",  
        "foregroundImage": "./assets/android-icon-foreground.png",  
        "backgroundImage": "./assets/android-icon-background.png",  
        "monochromeImage": "./assets/android-icon-monochrome.png"  
      },  
      "predictiveBackGestureEnabled": false,  
      "package": "com.ilkoadamov.adhan",  
      "permissions": [  
        "android.permission.SCHEDULE_EXACT_ALARM"  
      ]  
    },  
    "web": {  
      "favicon": "./assets/favicon.png"  
    },  
    "plugins": [  
      "expo-router",  
      [  
        "expo-splash-screen",  
        {  
          "backgroundColor": "#0F1B2D",  
          "image": "./assets/splash-icon.png",  
        }  
      ]  
    ]  
  }  
}
```

```

        "imageWidth": 160
    }
  ],
  "expo-font",
  [
    "expo-build-properties",
    {
      "android": {
        "cmakeVersion": "3.31.6"
      }
    }
  ],
  [
    "expo-localization",
    {
      "supportedLocales": {
        "ios": [
          "en",
          "bg"
        ],
        "android": [
          "en",
          "bg"
        ]
      }
    }
  ],
  [
    "expo-location",
    {
      "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place where you are.",
      "locationAlwaysAndWhenInUsePermission": false,
      "locationAlwaysPermission": false,
      "motionUsagePermission": false
    }
  ],
  "expo-background-task",
  [
    "expo-notifications",
    {
      "icon": "./assets/notification-icon.png",
      "color": "#D4A857",
      "sounds": [
        "./assets/sounds/ezan_chime.wav",
        "./assets/sounds/ezan_adhan.wav"
      ]
    }
  ]
],
"scheme": "adhan",
"backgroundColor": "#0F1B2D",
"experiments": {
  "typedRoutes": true
},
"locales": {
  "bg": "./locales/bg.json",
  "en": "./locales/en.json"
}
}
}

```

- Не ползваме `USE_EXACT_ALARM` : Google Play го разрешава само на будилници и календари. `SCHEDULE_EXACT_ALARM` е за всички, но от Android 14 потребителят го включва сам – приложението показва бутон (стъпка 7).

Стъпка 3

Звуци и иконка

Три нови файла в `assets`.

ФАЙЛ	КАКВО Е	РАЗМЕР
assets/sounds/ezan_chime.wav	кратък сигнал – две меки камбанки (1,9 сек.)	82 KB
assets/sounds/ezan_adhan.wav	временен „езан“ – мелодия в макам Хиджаз (27,8 сек.)	1,2 MB
assets/notification-icon.png	полумесец и звезда, бели на прозрачно, 96×96	1 KB

- Имената са с малки букви и долна черта – Android не приема друго в `res/raw`.
- WAV (PCM) работи и на двете платформи. iPhone пуска звук на известие само до 30 сек. – затова временният езан е 27,8 сек.
- Истинските записи на езан (с лиценз) идват в етап 6. Тогава само се сменят тези два файла.
- Иконката за лентата с известия на Android трябва да е само бяла с прозрачност – системата я оцветява.

Стъпка 4

Native модул „AdhanNative“ (Kotlin, само Android)

От Android 12 точните известия изискват разрешението „Аларми и напомняния“, а от Android 14 то е изключено по подразбиране. Без него известието може да закъснее с няколко минути. JavaScript не може да провери това сам – затова малък локален модул. В етап 5 в него идват алармата с пълния езан и местоположението без Google.

`expo-module.config.json` `modules/adhan-native`

- Само за Android. Модулите в папка `modules/` се свързват автоматично (autolinking) при build.

modules/adhan-native/expo-module.config.json6 реда

```
{
  "platforms": ["android"],
  "android": {
    "modules": ["expo.modules.adhannative.AdhanNativeModule"]
  }
}
```

build.gradle modules/adhan-native/android

- Стандартният Gradle файл на Expo модул.

modules/adhan-native/android/build.gradle

18 реда

```
plugins {
    id 'com.android.library'
    id 'expo-module-gradle-plugin'
}

group = 'expo.modules.adhannative'
version = '0.1.0'

android {
    namespace "expo.modules.adhannative"
    defaultConfig {
        versionCode 1
        versionName "0.1.0"
    }
    lintOptions {
        abortOnError false
    }
}
```

AdhanNativeModule.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- `canScheduleExactAlarms` – `AlarmManager.canScheduleExactAlarms()` ; до Android 12 – винаги `true` .
- `openExactAlarmSettings` – отваря системния екран „Аларми и напомняния“ точно за Езан; ако производителят го няма – информацията за приложението.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AdhanNativeModule.kt

57 реда

```
package expo.modules.adhannative

import android.app.AlarmManager
import android.content.ActivityNotFoundException
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.provider.Settings
import expo.modules.kotlin.exception.Exceptions
import expo.modules.kotlin.modules.Module
import expo.modules.kotlin.modules.ModuleDefinition

/**
 * Малкият native модул на Езан (само Android).
 * Етап 4: точни известия (Android 12+). В етап 5 тук идват алармата с пълния езан
 * и местоположението без Google услуги.
 */
class AdhanNativeModule : Module() {
    private val context: Context
        get() = appContext.reactContext ?: throw Exceptions.ReactContextLost()

    override fun definition() = ModuleDefinition {
        Name("AdhanNative")

        // Може ли приложението да планира точни аларми. До Android 12 – винаги да.
    }
```

```

// От Android 14 разрешението „Аларми и напомняния“ е изключено по подразбиране.
Function("canScheduleExactAlarms") {
  if (Build.VERSION.SDK_INT < Build.VERSION_CODES.S) {
    true
  } else {
    val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
    alarmManager.canScheduleExactAlarms()
  }
}

// Отваря системния екран „Аларми и напомняния“ за Езан.
// Ако производителят го е махнал – екрана с информация за приложението.
Function("openExactAlarmSettings") {
  val packageUri = Uri.parse("package:${context.packageName}")
  val intent =
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
      Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM, packageUri)
    } else {
      Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
    }
  intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
  try {
    context.startActivity(intent)
  } catch (e: ActivityNotFoundException) {
    context.startActivity(
      Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
packageUri).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    )
  }
}
}
}
}

```

index.ts modules/adhan-native

- `requireOptionalNativeModule` : на iPhone и в уеб модулът липсва и функциите връщат `null`, вместо приложението да спре.

modules/adhan-native/index.ts

27 реда

```

import { requireOptionalNativeModule } from 'expo';
import { Platform } from 'react-native';

/**
 * Native функциите на Езан (само Android). На iPhone и в уеб модулът липсва –
 * тогава функциите връщат безопасни стойности.
 */
interface AdhanNativeModule {
  canScheduleExactAlarms(): boolean;
  openExactAlarmSettings(): void;
}

const native = Platform.OS === 'android' ? requireOptionalNativeModule<AdhanNativeModule>('AdhanNative') :
null;

/** null – неприложимо (iPhone, уеб) или модулът липсва (стар build без него). */
export function canScheduleExactAlarms(): boolean | null {
  if (!native) return null;
  try {
    return native.canScheduleExactAlarms();
  }
}

```

```

    } catch {
      return null;
    }
  }

  export function openExactAlarmSettings(): void {
    native?.openExactAlarmSettings();
  }

```

Стъпка 5

Планът на известията

Чиста функция: подаваш часа, настройките и камбанките – получаващ списък от известия. Не знае нищо за телефона, затова е лесна за тестване.

notifications.ts src/domain

- За всеки от следващите 7 дни и всяка молитва с включена камбанка: известие в часа и (ако има) напомняне X мин. по-рано. Часовете са от същата `computeDay` като „Днес“ – винаги съвпадат.
- Изгревът е само известие, без езан, с текст „Краят на времето за Фаджр“.
- **До 62 известия:** iPhone пази най-много 64 чакащи за приложение; 2 остават за пробното. С напомняния това са ~5 дни. Ако не се събират всички, последното е „Отвори Езан“ – иначе на iPhone, ако приложението не се отваря и фоновата задача не тръгне, известията биха спрели тихо.
- Стабилни идентификатори (`ezan-20261001-maghrib-prayer`) – по тях службата разбира кое вече е планирано.

src/domain/notifications.ts

159 реда

```

import { formatHM } from './format';
import { PRAYER_IDS, PRAYERS, type PrayerId } from './prayers';
import { computeDay, type TimesOptions } from './times';

/**
 * Планът на известията – чиста функция, без expo-notifications, за да се тества.
 * Службата (src/services/notifications.ts) само го изпраща към телефона.
 */

/** Режимът на камбанката (виж src/store/alertPrefs.ts). */
export type AlertMode = 'off' | 'notify' | 'adhan';

/** Кой звук: кратък сигнал или езан (в етап 4 – временен звук). */
export type SoundKind = 'chime' | 'adhan';

export type NotificationKind = 'prayer' | 'reminder' | 'refresh';

export interface PlannedNotification {
  /** Стабилен идентификатор: „ezan-20261001-maghrib-prayer“. */
  id: string;
  at: Date;
  kind: NotificationKind;
  prayer: PrayerId | null;
  sound: SoundKind;
  title: string;
  body: string;
}

```

```

/** Текстовете на известията (идват от src/i18n/strings.ts). */
export interface NotificationTexts {
  prayers: Record<PrayerId, string>;
  prayerTitle: (prayer: string, time: string) => string;
  /** „Време е за Иша (العشاء) · София“ */
  prayerBody: (prayer: string, arabic: string, place: string) => string;
  sunriseBody: (place: string) => string;
  /** Вторият ред: „Следваща: Фаджр в 05:37 (утре)“ */
  nextLine: (prayer: string, time: string, tomorrow: boolean) => string;
  reminderTitle: (prayer: string, minutes: number, time: string) => string;
  reminderBody: (place: string) => string;
  refreshTitle: string;
  refreshBody: string;
}

export interface PlanInput {
  now: Date;
  options: TimesOptions;
  modes: Record<PrayerId, AlertMode>;
  /** Напомняне X минути преди молитвата; 0 = без напомняне. */
  reminderMinutes: number;
  placeName: string;
  texts: NotificationTexts;
  /**
   * Колко известия най-много. iOS пази до 64 чакащи известия на приложение;
   * оставяме място за пробното известие.
   */
  limit?: number;
  /** За колко дни напред (от днес). */
  days?: number;
}

export const NOTIFICATION_LIMIT = 62;
export const PLAN_DAYS = 7;
/** Известие по-рано от това (напр. след 2 сек.) не се планира – няма смисъл. */
const MIN_LEAD_MS = 5_000;

const pad = (n: number) => String(n).padStart(2, '0');
const ymd = (d: Date) => `${d.getFullYear()}${pad(d.getMonth() + 1)}${pad(d.getDate())}`;

/**
 * Всички известия за следващите дни, по ред на часа, до `limit`.
 * Ако не се събират всички, последното място е за напомняне „отвори приложението“ –
 * на iPhone без него известията биха спрели тихо след ~5 дни, ако приложението не се отваря.
 */
export function planNotifications(input: PlanInput): PlannedNotification[] {
  const { now, options, modes, reminderMinutes, placeName, texts } = input;
  const limit = input.limit ?? NOTIFICATION_LIMIT;
  const days = input.days ?? PLAN_DAYS;
  const earliest = now.getTime() + MIN_LEAD_MS;

  const dayAt = (d: number) => new Date(now.getFullYear(), now.getMonth(), now.getDate() + d, 12);
  const all: PlannedNotification[] = [];
  // денят след последния е нужен само за реда „Следваща: Фаджр ... (утре)“
  let next = computeDay(dayAt(0), options);
  for (let d = 0; d < days; d++) {
    const date = dayAt(d);
    const day = next;
    next = computeDay(dayAt(d + 1), options);
    PRAYER_IDS.forEach((id, i) => {
      const mode = modes[id];

```

```

    if (mode === 'off') return;
    const time = day[i].time;
    const name = texts.prayers[id];
    const hm = formatHM(time);
    const key = `ezan-${ymd(date)}-${id}`;
    const following = i + 1 < day.length ? day[i + 1] : next[0];
    const nextLine = texts.nextLine(
      texts.prayers[following.id],
      formatHM(following.time),
      i + 1 >= day.length,
    );

    if (reminderMinutes > 0) {
      all.push({
        id: `${key}-reminder`,
        at: new Date(time.getTime() - reminderMinutes * 60_000),
        kind: 'reminder',
        prayer: id,
        sound: 'chime',
        title: texts.reminderTitle(name, reminderMinutes, hm),
        body: texts.reminderBody(placeName),
      });
    }
    all.push({
      id: `${key}-prayer`,
      at: time,
      kind: 'prayer',
      prayer: id,
      // изгревът е само известие, без езан (виж PRAYERS.sunrise.canAlarm)
      sound: mode === 'adhan' && id !== 'sunrise' ? 'adhan' : 'chime',
      title: texts.prayerTitle(name, hm),
      body:
        (id === 'sunrise' ? texts.sunriseBody(placeName) : texts.prayerBody(name, PRAYERS[id].arabic,
        placeName)) +
        '\n' +
        nextLine,
    });
  });
}

const upcoming = all
  .filter((n) => !isNaN(n.at.getTime()) && n.at.getTime() >= earliest)
  .sort((a, b) => a.at.getTime() - b.at.getTime());

if (upcoming.length <= limit) return upcoming;

const kept = upcoming.slice(0, limit - 1);
const last = kept[kept.length - 1];
kept.push({
  id: 'ezan-refresh',
  at: new Date(last.at.getTime() + 2 * 60_000),
  kind: 'refresh',
  prayer: null,
  sound: 'chime',
  title: texts.refreshTitle,
  body: texts.refreshBody,
});
return kept;
}

/**

```

```

* „Отпечатак“ на плана: ако е същият като вече изпратения, не пренасрочваме
* (иначе всяко отваряне на приложението би трило и създавало 60 известия).
*/
export function planSignature(plan: PlannedNotification[], extra: string): string {
  return (
    extra +
    '|' +
    plan.map((n) => `${n.id}@${n.at.getTime()}:${n.sound}:${n.title}:${n.body}`).join('|')
  );
}

```

src/i18n/strings.ts – текстовете на известията (и на английски)

```

notifications: {
  prayerTitle: (prayer, time) => `${prayer} - ${time}`, // „Иша - 20:41“
  prayerBody: (prayer, arabic, place) => `Време е за намаза ${prayer} (${arabic}) · ${place}`,
  nextLine: (prayer, time, tomorrow) => `Следваща: ${prayer} в ${time}${tomorrow ? ' (утре)' : ''}`,
  sunriseBody: (place) => `Изгрев - края на времето за Фаджр · ${place}`,
  reminderTitle: (prayer, minutes, time) => `${prayer} след ${minutes} мин. - ${time}`,
  reminderBody: (place) => `Време е за абдест и подготовка за намаза · ${place}`,
  refreshTitle: 'Отвори Езан',
  ...
}

```

Пример в разгънато известие: **Иша - 20:41** / Време е за намаза Иша (العشاء) · София / Следваща: Фаджр в 05:37 (утре).

alertPrefs.ts src/store

- Типът `AlertMode` вече е в `domain/notifications.ts` – едно място за двете части.

src/store/alertPrefs.ts

49 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import type { AlertMode } from '@domain/notifications';
import { PRAYERS, type PrayerId } from '@domain/prayers';

/** Трите състояния на камбанката: сиво / бяло / злато. */
export type { AlertMode };

const DEFAULT_MODES: Record<PrayerId, AlertMode> = {
  fajr: 'adhan',
  sunrise: 'notify',
  dhuhr: 'notify',
  asr: 'adhan',
  maghrib: 'adhan',
  isha: 'notify',
};

/** off → notify → adhan → off. Изгрев прескача „езан“. */
export function nextMode(id: PrayerId, mode: AlertMode): AlertMode {
  if (mode === 'off') return 'notify';
  if (mode === 'notify') return PRAYERS[id].canAlarm ? 'adhan' : 'off';
  return 'off';
}

interface AlertPrefsState {

```

```

modes: Record<PrayerId, AlertMode>;
/** Сменя режима и връща новия (за toast-a). */
cycle: (id: PrayerId) => AlertMode;
}

export const useAlertPrefs = create<AlertPrefsState>()(
  persist(
    (set, get) => ({
      modes: DEFAULT_MODES,
      cycle: (id) => {
        const mode = nextMode(id, get().modes[id]);
        set((s) => ({ modes: { ...s.modes, [id]: mode } }));
        return mode;
      },
    }),
    {
      name: 'ezan.alert-prefs',
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (s) => ({ modes: s.modes }),
    },
  ),
);

```

Стъпка 6

Службата: разрешение, канали, планиране

Единственото място, което говори с `expo-notifications`.

`notifications.ts` `src/services`

- **Канали (Android):** „Молитви“ (кратък сигнал), „Езан“ (алармата, най-висок приоритет), „Напомняния“. Звукът и вибрацията на канал не могат да се сменят след създаването му – затова каналите с и без вибрация са различни (`adhan-v3` / `adhan-s3`), а ненужният се трие.
- **„Езан“ звучи като аларма** (`AndroidAudioUsage.ALARM`): през потока за аларми, не зависи от силата на звука за известия и от безшумния режим. Промяната на канала изисква нов идентификатор – затова версия 2; каналите от версия 1 се трият.
- **Вибрация (версия 3 на каналите):** при „Езан“ – 1 сек. / 0,7 сек. пауза, докато звучи езанът (~28 сек.); при известие и напомняне – едно вибриране от 1 сек. Силата се определя от телефона; в етап 5 алармата ще управлява вибрацията сама, с пълна сила.
- **Звук:** името на файла се подава и на Android – без него библиотеката маркира известието като тихо и каналът не звъни.
- **Разрешение:** на iPhone важи `ios.status` (и „временното“ разрешение се брои); на Android 13+ – системният въпрос. Ако е отказано завинаги, бутонът отваря настройките на телефона.
- **Пренасрочване без излишна работа:** всяко известие носи „отпечатък“ (час, звук, канал, текст, точен/неточен). Службата трие само променените и добавя само новите – отварянето на приложението не трие и не създава 60 известия всеки път.
- Паралелните извиквания се обединяват в едно.

- Пробното известие е след 10 сек., с избрания звук, и не се трие при пренасрочване.

src/services/notifications.ts

328 реда

```
import * as Notifications from 'expo-notifications';
import { Linking, Platform } from 'react-native';
import { create } from 'zustand';

import { canScheduleExactAlarms, openExactAlarmSettings } from '../../modules/adhan-native';

import { planNotifications, type PlannedNotification, type SoundKind } from '@domain/notifications';
import { getI18n } from '@i18n';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, selectTimesOptions, useSettings } from '@store/settings';

/**
 * Известията за намаз – само локални (без интернет, без Firebase/FCM и без HMS Push).
 * Android планира всяко известие с AlarmManager, iPhone – с UNUserNotificationCenter.
 * Затова работят еднакво на телефони с Google услуги, на Huawei без тях и на iPhone.
 */

export type NotificationPermission = 'granted' | 'denied' | 'undetermined' | 'unavailable';

interface NotificationStatus {
  /** false, докато разрешението още не е проверено (първите мигове след стартиране). */
  checked: boolean;
  permission: NotificationPermission;
  /** Android 13+/iOS: може ли приложението пак да покаже системния въпрос. */
  canAskAgain: boolean;
  /** Android 12+: точни известия („Аларми и напомняния“); null – неприложимо. */
  exact: boolean | null;
  /** Колко известия са планирани и до кога. */
  count: number;
  until: Date | null;
}

export const useNotificationStatus = create<NotificationStatus>(() => ({
  checked: false,
  permission: Platform.OS === 'web' ? 'unavailable' : 'undetermined',
  canAskAgain: true,
  exact: null,
  count: 0,
  until: null,
}));

const SUPPORTED = Platform.OS === 'android' || Platform.OS === 'ios';

/** Файловете от assets/sounds (виж app.json → expo-notifications → sounds). */
const SOUND_FILE: Record<SoundKind, string> = {
  chime: 'ezan_chime.wav',
  adhan: 'ezan_adhan.wav',
};

/* ----- канали (Android) */

type ChannelKind = 'prayer' | 'adhan' | 'reminder';

/**
 * На Android звукът и вибрацията са на канала и не могат да се сменят след създаването му.
 * Затова каналите с и без вибрация са различни („adhan-v2“ / „adhan-s2“),
 * а ненужният се трие при смяна на настройката.
 */
```

```

* Версия 2: каналът „Езан“ звучи като аларма (виж setupChannels).
* Версия 3: при „Езан“ вибрацията продължава, докато звучи езанът; при известията – една.
* Каналите от по-старите версии се трият.
*/
const CHANNEL_VERSION = 3;
function channelId(kind: ChannelKind, vibrate: boolean, version = CHANNEL_VERSION): string {
    return `${kind}-${vibrate ? 'v' : 's'}${version}`;
}

/** Известие и напомняне: едно дълго вибриране (1 сек.). */
const VIBRATION_ONCE = [0, 1000];

/**
* Езан: 1 сек. вибрация / 0,7 сек. пауза, докато звучи временният езан (~28 сек.).
* Силата на вибрацията се определя от телефона (Настройки → Звуци и вибрация).
* В етап 5 алармата управлява вибрацията сама – със силата и продължителността на пълния езан.
*/
const VIBRATION_ADHAN = [0, ...Array.from({ length: 16 }, () => [1000, 700]).flat()];

const vibrationFor = (kind: ChannelKind) => (kind === 'adhan' ? VIBRATION_ADHAN : VIBRATION_ONCE);

function channelFor(n: PlannedNotification): ChannelKind {
    if (n.kind === 'reminder' || n.kind === 'refresh') return 'reminder';
    return n.sound === 'adhan' ? 'adhan' : 'prayer';
}

let channelsFor: boolean | null = null;

async function setupChannels(vibrate: boolean): Promise<void> {
    if (Platform.OS !== 'android' || channelsFor === vibrate) return;
    const { t } = getI18n();
    const { AndroidAudioUsage: Usage, AndroidAudioContentType: Content, AndroidImportance: Imp } =
Notifications;
    const defs: {
        kind: ChannelKind;
        name: string;
        sound: SoundKind;
        importance: Notifications.AndroidImportance;
        usage: Notifications.AndroidAudioUsage;
    }[] = [
        { kind: 'prayer', name: t.notifications.channelPrayer, sound: 'chime', importance: Imp.HIGH, usage:
Usage.NOTIFICATION },
        // Езанът звучи през потока за аларми: не зависи от силата на звука за известия
        // и от безшумния режим – както будилник. (Пълният езан с „Спри“ – етап 5.)
        { kind: 'adhan', name: t.notifications.channelAdhan, sound: 'adhan', importance: Imp.MAX, usage:
Usage.ALARM },
        { kind: 'reminder', name: t.notifications.channelReminder, sound: 'chime', importance: Imp.HIGH, usage:
Usage.NOTIFICATION },
    ];
    for (const d of defs) {
        await Notifications.setNotificationChannelAsync(channelId(d.kind, vibrate), {
            name: d.name,
            importance: d.importance,
            sound: SOUND_FILE[d.sound],
            audioAttributes: { usage: d.usage, contentType: Content.SONIFICATION },
            enableVibrate: vibrate,
            vibrationPattern: vibrate ? vibrationFor(d.kind) : null,
            lockscreenVisibility: Notifications.AndroidNotificationVisibility.PUBLIC,
        });
        await Notifications.deleteNotificationChannelAsync(channelId(d.kind, !vibrate));
        // каналите от по-стари версии

```

```

    for (let v = 1; v < CHANNEL_VERSION; v++) {
        await Notifications.deleteNotificationChannelAsync(channelId(d.kind, true, v));
        await Notifications.deleteNotificationChannelAsync(channelId(d.kind, false, v));
    }
}
channelsFor = vibrate;
}

/* ----- разрешение */

function toPermission(p: Notifications.NotificationPermissionsStatus): NotificationPermission {
    if (Platform.OS === 'ios') {
        const s = p.ios?.status;
        if (
            s === Notifications.IosAuthorizationStatus.AUTHORIZED ||
            s === Notifications.IosAuthorizationStatus.PROVISIONAL ||
            s === Notifications.IosAuthorizationStatus.EPHEMERAL
        ) {
            return 'granted';
        }
        return s === Notifications.IosAuthorizationStatus.DENIED ? 'denied' : 'undetermined';
    }
    if (p.granted) return 'granted';
    return p.status === 'undetermined' ? 'undetermined' : 'denied';
}

export async function refreshPermission(): Promise<NotificationPermission> {
    if (!SUPPORTED) return 'unavailable';
    const p = await Notifications.getPermissionsAsync();
    const permission = toPermission(p);
    useNotificationStatus.setState({
        checked: true,
        permission,
        canAskAgain: p.canAskAgain,
        exact: canScheduleExactAlarms(),
    });
    return permission;
}

/**
 * Пита за разрешение. Ако системата вече не показва въпроса (отказано завинаги) -
 * отваря настройките на приложението в телефона.
 */
export async function requestPermission(): Promise<NotificationPermission> {
    if (!SUPPORTED) return 'unavailable';
    const current = await Notifications.getPermissionsAsync();
    if (toPermission(current) === 'denied' && !current.canAskAgain) {
        await Linking.openSettings();
        return 'denied';
    }
    const p = await Notifications.requestPermissionsAsync({
        ios: { allowAlert: true, allowSound: true, allowBadge: false },
    });
    const permission = toPermission(p);
    useNotificationStatus.setState({ permission, canAskAgain: p.canAskAgain });
    if (permission === 'granted') rescheduleNotifications();
    return permission;
}

export { openExactAlarmSettings };

```

```

/* ----- планиране */

/** Всичко, което определя едно известие. Ако се смени, известието се пренасрочва. */
function signatureOf(n: PlannedNotification, channel: string, exact: boolean | null): string {
    return `${n.at.getTime()}|${n.sound}|${channel}|${exact}|${n.title}|${n.body}`;
}

let running: Promise<void> | null = null;
let again = false;

/**
 * Привежда планираните известия в съответствие с настройките.
 * Вика се при стартиране, при връщане в приложението, при всяка промяна на мястото,
 * часовете, камбанките, напомнянето и вибрацията, и от фоновата задача.
 * Паралелните извиквания се обединяват.
 */
export function rescheduleNotifications(): Promise<void> {
    if (running) {
        again = true;
        return running;
    }
    running = doReschedule()
        .catch((e) => console.warn('[notifications]', e))
        .finally(() => {
            running = null;
            if (again) {
                again = false;
                rescheduleNotifications();
            }
        });
    return running;
}

async function doReschedule(): Promise<void> {
    if (!SUPPORTED) return;
    const permission = await refreshPermission();
    const exact = useNotificationStatus.getState().exact;

    const s = useSettings.getState();
    const { lang, t } = getI18n();
    const plan =
        permission === 'granted'
        ? planNotifications({
            now: new Date(),
            options: selectTimesOptions(s).options,
            modes: useAlertPrefs.getState().modes,
            reminderMinutes: s.reminderMinutes,
            placeName: selectLocation(s).names[lang],
            texts: { prayers: t.prayers, ...t.notifications },
        })
        : [];

    await setupChannels(s.vibrate);

    // Сравнява с вече планираните: трие само променените и липсващите, добавя само новите.
    const existing = await Notifications.getAllScheduledNotificationsAsync();
    const ours = new Map(
        existing
            .filter((r) => r.identifier.startsWith('ezan-'))
            .map((r) => [r.identifier, (r.content.data as { sig?: string } | null)?.sig ?? '']),
    );

```

```

const wanted = plan.map((n) => {
  const channel = channelId(channelFor(n), s.vibrate);
  return { n, channel, sig: signatureOf(n, channel, exact) };
});
const wantedIds = new Set(wanted.map((w) => w.n.id));

for (const [id, sig] of ours) {
  const w = wanted.find((x) => x.n.id === id);
  if (!wantedIds.has(id) || !w || w.sig !== sig) {
    await Notifications.cancelScheduledNotificationAsync(id);
    ours.delete(id);
  }
}

for (const { n, channel, sig } of wanted) {
  if (ours.has(n.id)) continue;
  await Notifications.scheduleNotificationAsync({
    identifier: n.id,
    content: {
      title: n.title,
      body: n.body,
      // на Android звукът е от канала, но без име на звук известието би било „тихо“
      sound: SOUND_FILE[n.sound],
      // за Android под 8; на по-новите вибрацията е от канала
      vibrate: s.vibrate ? vibrationFor(channelFor(n)) : undefined,
      data: { sig, kind: n.kind, prayer: n.prayer, sound: n.sound, at: n.at.getTime() },
      ...(Platform.OS === 'android' ? { color: '#D4A857' } : {}),
    },
    trigger: {
      type: Notifications.SchedulableTriggerInputTypes.DATE,
      date: n.at,
      channelId: channel,
    },
  });
}

useNotificationStatus.setState({
  count: plan.length,
  until: plan.length ? plan[plan.length - 1].at : null,
});
}

/** Трие всички известия на Езан и ги планира наново (от „Проверка на известията“). */
export async function resetAllNotifications(): Promise<void> {
  if (!SUPPORTED) return;
  await running;
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  for (const r of existing) {
    if (r.identifier.startsWith('ezan-')) await
Notifications.cancelScheduledNotificationAsync(r.identifier);
  }
  channelsFor = null;
  await rescheduleNotifications();
}

/** Пробно известие след 10 сек. - с кратък сигнал или с езан. Връща часа, в който ще дойде. */
export async function sendTestNotification(sound: SoundKind): Promise<number> {
  const at = Date.now() + 10_000;
  if (!SUPPORTED) return at;
  const s = useSettings.getState();

```

```

await setupChannels(s.vibrate);
const { t } = getI18n();
await Notifications.scheduleNotificationAsync({
  identifier: `test-${Date.now()}`,
  content: {
    title: t.notifications.testTitle,
    body: t.notifications.testBody,
    sound: SOUND_FILE[sound],
    vibrate: s.vibrate ? vibrationFor(sound === 'adhan' ? 'adhan' : 'prayer') : undefined,
    data: { kind: 'test', sound, at },
  },
  trigger: {
    type: Notifications.SchedulableTriggerInputTypes.DATE,
    date: new Date(at),
    channelId: channelId(sound === 'adhan' ? 'adhan' : 'prayer', s.vibrate),
  },
});
return at;
}

/** Как се показва известие, докато приложението е отворено: като обикновено, със звук. */
export function configureNotificationHandler(): void {
  if (!SUPPORTED) return;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({
      shouldShowBanner: true,
      shouldShowList: true,
      shouldPlaySound: true,
      shouldSetBadge: false,
    }),
  });
}

```

backgroundTask.ts src/services

- Фонова задача на ~6 часа допланира следващите дни. Android – WorkManager (не зависи от Google), iPhone – BGTaskScheduler (системата решава кога; обикновено нощем, на зарядно).
- Задачата е дефинирана при зареждане на кода: системата може да пусне приложението само за нея. Тогава първо четете настройките от паметта (`rehydrate`).

src/services/backgroundTask.ts

43 реда

```

import * as BackgroundTask from 'expo-background-task';
import * as TaskManager from 'expo-task-manager';
import { Platform } from 'react-native';

import { useAlertPrefs } from '@store/alertPrefs';
import { useSettings } from '@store/settings';

import { rescheduleNotifications } from './notifications';

/**
 * Фонова задача: веднъж на няколко часа допланира известията за следващите дни,
 * дори ако приложението не се отваря. Android – WorkManager (работи и без Google услуги),
 * iPhone – BGTaskScheduler (системата решава кога; обикновено нощем, на зарядно).
 */
export const RESCHEDULE_TASK = 'ezan-reschedule-notifications';

// Задачата трябва да е дефинирана при зареждане на кода (не в компонент) -

```

```
// системата може да пусне приложението само за нея, без да показва екран.
if (Platform.OS !== 'web') {
  TaskManager.defineTask(RESCHEDULE_TASK, async () => {
    try {
      // при пускане само за задачата настройките още не са прочетени от паметта
      await Promise.all([useSettings.persist.rehydrate(), useAlertPrefs.persist.rehydrate()]);
      await rescheduleNotifications();
      return BackgroundTask.BackgroundTaskResult.Success;
    } catch {
      return BackgroundTask.BackgroundTaskResult.Failed;
    }
  });
}

export async function registerBackgroundReschedule(): Promise<void> {
  if (Platform.OS === 'web') return;
  try {
    const status = await BackgroundTask.getStatusAsync();
    if (status !== BackgroundTask.BackgroundTaskStatus.Available) return;
    if (!(await TaskManager.isTaskRegisteredAsync(RESCHEDULE_TASK))) {
      await BackgroundTask.registerTaskAsync(RESCHEDULE_TASK, { minimumInterval: 6 * 60 }); // минути
    }
  } catch (e) {
    console.warn('[background-task]', e);
  }
}
```

useNotificationScheduler.ts src/hooks

- При първо пускане пита за разрешение (защото камбанките по подразбиране са включени), после планира и регистрира фоновата задача.
- Слуша настройките и камбанките – пренасрочва 0,6 сек. след последната промяна (за да не планира при всяко „+“). При връщане в приложението – също, така хваща и разрешението „Аларми и напомняния“.

src/hooks/useNotificationScheduler.ts

47 реда

```
import { useEffect } from 'react';
import { AppState } from 'react-native';

import { registerBackgroundReschedule } from '@services/backgroundTask';
import { refreshPermission, requestPermission, rescheduleNotifications } from '@services/notifications';
import { useAlertPrefs } from '@store/alertPrefs';
import { useSettings } from '@store/settings';

/**
 * Пуска се веднъж в корена на приложението, след като настройките са заредени.
 * - при първо пускане пита за разрешение за известия;
 * - планира известията и ги пренасрочва при всяка промяна на мястото, часовете,
 * камбанките, напомнянето и вибрацията (с малко изчакване, за да не се планира
 * при всяко натискане на „+“);
 * - при връщане в приложението (и след „Аларми и напомняния“ в настройките на телефона);
 * - регистрира фоновата задача.
 */
export function useNotificationScheduler(ready: boolean) {
  useEffect(() => {
    if (!ready) return;

    (async () => {
```

```

    const permission = await refreshPermission();
    if (permission === 'undetermined') await requestPermission();
    await rescheduleNotifications();
    await registerBackgroundReschedule();
  })();

  let timer: ReturnType<typeof setTimeout> | undefined;
  const soon = () => {
    clearTimeout(timer);
    timer = setTimeout(rescheduleNotifications, 600);
  };
  const unsubSettings = useSettings.subscribe(soon);
  const unsubAlerts = useAlertPrefs.subscribe(soon);
  const sub = AppState.addEventListener('change', (state) => {
    if (state === 'active') rescheduleNotifications();
  });

  return () => {
    clearTimeout(timer);
    unsubSettings();
    unsubAlerts();
    sub.remove();
  };
}, [ready]);
}

```

`_layout.tsx` src/app

- Импортира фоновата задача, настройва показването на известие при отворено приложение (като обикновено, със звук) и пуска `useNotificationScheduler`, след като настройките са заредени.

src/app/_layout.tsx

92 реда

```

import { Amiri_400Regular, Amiri_700Bold } from '@expo-google-fonts/amiri';
import {
  Manrope_400Regular,
  Manrope_500Medium,
  Manrope_600SemiBold,
  Manrope_700Bold,
  Manrope_800ExtraBold,
} from '@expo-google-fonts/manrope';
import { useFonts } from 'expo-font';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect, useState } from 'react';

import { useLocationUpdater } from '@hooks/useLocationUpdater';
import { useNotificationScheduler } from '@hooks/useNotificationScheduler';
// дефинира фоновата задача при зареждане на кода (виж файла)
import '@services/backgroundTask';
import { configureNotificationHandler } from '@services/notifications';
import { useAlertPrefs } from '@store/alertPrefs';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';

// Splash екранът стои, докато шрифтовете и запазените настройки се заредят -
// иначе за миг би се показвала София по подразбиране вместо твоето място.
SplashScreen.preventAutoHideAsync();
// известие, дошло докато приложението е отворено, се показва като обикновено - със звук

```

```

configureNotificationHandler();

function useStoresHydrated(): boolean {
  const [done, setDone] = useState(
    () => useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated(),
  );
  useEffect(() => {
    const check = () => setDone(useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated());
    const a = useSettings.persist.onFinishHydration(check);
    const b = useAlertPrefs.persist.onFinishHydration(check);
    check();
    // предпазител: ако четенето от паметта на телефона се провали, zustand не съобщава
    // „готово“ – след 2 сек. приложението се показва с настройките по подразбиране
    const timer = setTimeout(() => setDone(true), 2000);
    return () => {
      a();
      b();
      clearTimeout(timer);
    };
  }, []);
  return done;
}

export default function RootLayout() {
  const [loaded, error] = useFonts({
    Manrope_400Regular,
    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
    Amiri_400Regular,
    Amiri_700Bold,
  });

  const hydrated = useStoresHydrated();
  const ready = (loaded || !error) && hydrated;
  useNotificationScheduler(hydrated);

  useEffect(() => {
    if (ready) SplashScreen.hideAsync();
  }, [ready]);

  // GPS при стартиране и при връщане в приложението (ако мястото е автоматично)
  useLocationUpdater();

  if (!ready) return null;

  return (
    <>
      <StatusBar style="light" />
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: { backgroundColor: colors.base },
        }}
      >
        <Stack.Screen name="tabs" />
        {/* изборът на място се отваря отдолу нагоре като модален екран */}
        <Stack.Screen name="place" options={{ presentation: 'modal' }} />
        <Stack.Screen name="about" />
        <Stack.Screen name="diagnostics" />
      </Stack>
    </>
  );
}

```

```

    </Stack>
  </>
);
}

```

settings.ts src/store

- Нови полета: `reminderMinutes` (0 – без, 5, 10, 15, 20, 30) и `vibrate`. В старите записи ги няма – тогава важат стойностите по подразбиране.
- `buildTimesOptions` / `selectTimesOptions` – опциите за часовете без React, за да ги ползват и известията във фонов режим.

src/store/settings.ts

156 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,
  DEFAULT_METHOD,
} from '@config/defaults';
import type { AsrMadhab, CalculationMethodId, HighLatRuleId } from '@domain/calc';
import type { AppLocation } from '@domain/location';
import type { PrayerId } from '@domain/prayers';
import type { TimesOptions } from '@domain/times';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни
/** Възможните напомняния преди молитвата (минути); 0 = без напомняне. */
export const REMINDER_CHOICES = [0, 5, 10, 15, 20, 30] as const;

const NO_OFFSETS: Record<PrayerId, number> = {
  fajr: 0,
  sunrise: 0,
  dhuhr: 0,
  asr: 0,
  maghrib: 0,
  isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
  /** true – мястото идва от GPS; false – избрано ръчно. */
  autoLocation: boolean;
  /** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
  gpsLocation: AppLocation | null;
  /** Ръчно избраното място. */
  manualLocation: AppLocation | null;

  method: CalculationMethodId;
  madhab: AsrMadhab;
  highLatRule: HighLatRuleId;
  offsets: Record<PrayerId, number>;
  hijriAdjust: number;
}

```

```

/** Напомняне X минути преди молитвата; 0 = без. */
reminderMinutes: number;
/** Вибрация при известие (Android; на iPhone решава телефонът). */
vibrate: boolean;

setAutoLocation: (on: boolean) => void;
setGpsLocation: (loc: AppLocation) => void;
chooseLocation: (loc: AppLocation) => void;
setMethod: (m: CalculationMethodId) => void;
setMadhab: (m: AsrMadhab) => void;
setHighLatRule: (r: HighLatRuleId) => void;
changeOffset: (id: PrayerId, delta: number) => void;
resetOffsets: () => void;
changeHijriAdjust: (delta: number) => void;
setReminderMinutes: (n: number) => void;
setVibrate: (on: boolean) => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
      hijriAdjust: 0,
      reminderMinutes: 0,
      vibrate: true,

      setAutoLocation: (on) => set({ autoLocation: on }),
      setGpsLocation: (loc) => set({ gpsLocation: loc }),
      // ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
      chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
      setMethod: (method) => set({ method }),
      setMadhab: (madhab) => set({ madhab }),
      setHighLatRule: (highLatRule) => set({ highLatRule }),
      changeOffset: (id, delta) =>
        set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
      resetOffsets: () => set({ offsets: NO_OFFSETS }),
      changeHijriAdjust: (delta) =>
        set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
      setReminderMinutes: (reminderMinutes) => set({ reminderMinutes }),
      setVibrate: (vibrate) => set({ vibrate }),
    })),
    {
      name: 'ezan.settings',
      version: 1,
      storage: createJSONStorage(() => AsyncStorage),
      // новите полета (reminderMinutes, vibrate) липсват в старите записи – тогава важат стойностите по подразбиране
      partialize: ({
        autoLocation,
        gpsLocation,
        manualLocation,
        method,
        madhab,
        highLatRule,
        offsets,

```

```

        hijriAdjust,
        reminderMinutes,
        vibrate,
    }) => ({
        reminderMinutes,
        vibrate,
        autoLocation,
        gpsLocation,
        manualLocation,
        method,
        madhab,
        highLatRule,
        offsets,
        hijriAdjust,
    }),
    },
),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {
    const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
    return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

type TimesParts = Pick<SettingsState, 'method' | 'madhab' | 'highLatRule' | 'offsets'> & { location:
AppLocation };

/** Опциите за часовете и техният „отпечатък“ `key` – по него се решава кога да се преизчисли. */
export function buildTimesOptions({ location, method, madhab, highLatRule, offsets }: TimesParts): {
    options: TimesOptions;
    key: string;
} {
    const options: TimesOptions = { location, method, madhab, highLatRule, offsets };
    const key = JSON.stringify([
        location.id,
        location.latitude,
        location.longitude,
        location.source,
        location.muftiShift,
        method,
        madhab,
        highLatRule,
        offsets,
    ]);
    return { options, key };
}

/** Същото, направо от състоянието – без React (ползва се и от известията във фонов режим). */
export function selectTimesOptions(s: SettingsState): { options: TimesOptions; key: string } {
    return buildTimesOptions({ ...s, location: selectLocation(s) });
}

```

useTimesOptions.ts src/hooks

- Ползва същото `buildTimesOptions` – „Днес“, „Месец“ и известията винаги имат едни и същи часове.

src/hooks/useTimesOptions.ts

26 реда

```
import { useMemo } from 'react';
import { useShallow } from 'zustand/react/shallow';

import type { TimesOptions } from '@domain/times';
import { buildTimesOptions, selectLocation, useSettings } from '@store/settings';

/**
 * Всичко, от което зависят часовете, събрано от настройките.
 * Ползва се от „Днес“ и „Месец“; известията ползват същото `buildTimesOptions`
 * (чрез `selectTimesOptions`), затова всички показват едни и същи часове.
 */
export function useTimesOptions(): { options: TimesOptions; key: string } {
  const { location, method, madhab, highLatRule, offsets } = useSettings(
    useShallow((s) => ({
      location: selectLocation(s),
      method: s.method,
      madhab: s.madhab,
      highLatRule: s.highLatRule,
      offsets: s.offsets,
    })),
  );
  return useMemo(
    () => buildTimesOptions({ location, method, madhab, highLatRule, offsets }),
    [location, method, madhab, highLatRule, offsets],
  );
}
```

index.ts src/i18n

- `getI18n()` – езикът и текстовете без React, за известията.

src/i18n/index.ts

34 реда

```
import { getLocales, useLocales } from 'expo-localization';

import { bg, en, type Strings } from './strings';

export type Lang = 'bg' | 'en';

const STRINGS: Record<Lang, Strings> = { bg, en };

/**
 * Правилото: ако основният език на телефона е български – български,
 * за всеки друг език – английски.
 */
export function resolveLang(languageCode: string | null | undefined): Lang {
  return languageCode?.toLowerCase() === 'bg' ? 'bg' : 'en';
}

/**
 * Текущият език и текстовете за него.
 * `useLocales()` прерисува компонента, когато потребителят смени езика на телефона
 * (или езика само за това приложение – Android 13+ / iOS).
 */
```

```

export function useI18n(): { lang: Lang; t: Strings } {
  const locales = useLocales();
  const lang = resolveLang(locales[0]?.languageCode);
  return { lang, t: STRINGS[lang] };
}

export type { Strings };

/** Същото без React – за известията, които се планират и във фонов режим. */
export function getI18n(): { lang: Lang; t: Strings } {
  const lang = resolveLang(getLocales()[0]?.languageCode);
  return { lang, t: STRINGS[lang] };
}

```

Стъпка 7

Екраните

Нова група „Известия“ в „Настройки“ и подсказка на „Днес“.

NotificationsGroup.tsx src/components/settings

- **Разрешение:** дадено / отказано / не е дадено, с бутон „Разреши известията“.
- **Точни известия** (само ако Android не ги позволява): обяснение и бутон „Разреши „Аларми и напомняния““.
- **Напомняне преди молитвата:** без / 5 / 10 / 15 / 20 / 30 мин.
- **Вибрация** – само на Android; на iPhone решават настройките на телефона.
- Ред „Планирани 38 известия – до Ср, 7 окт., 20:05“ под групата и ред „Проверка на известията“ (пробното известие е там).

src/components/settings/NotificationsGroup.tsx

114 реда

```

import { router } from 'expo-router';
import { useState } from 'react';
import { Platform, Pressable, StyleSheet, Text, View } from 'react-native';

import { formatGregorianShort, formatHM } from '@/domain/format';
import { useI18n } from '@/i18n';
import { openExactAlarmSettings, requestPermission, useNotificationStatus } from '@/services/notifications';
import { REMINDER_CHOICES, useSettings } from '@/store/settings';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

import { Choice, Group, NavRow, Row, SwitchRow, settingsStyles } from './controls';

/** Групата „Известия“ в „Настройки“: разрешение, точни известия, напомняне, вибрация, проба. */
export function NotificationsGroup() {
  const { t } = useI18n();
  const n = t.notifications;
  const status = useNotificationStatus();
  const reminderMinutes = useSettings((s) => s.reminderMinutes);
  const setReminderMinutes = useSettings((s) => s.setReminderMinutes);
  const vibrate = useSettings((s) => s.vibrate);
  const setVibrate = useSettings((s) => s.setVibrate);
  const [open, setOpen] = useState(false);

```

```

if (status.permission === 'unavailable') {
    return (
        <Group label={n.group}>
            <Row first>
                <Text style={styles.text}>{n.webNote}</Text>
            </Row>
        </Group>
    );
}

const granted = status.permission === 'granted';
const permissionText = granted ? n.granted : status.permission === 'denied' ? n.denied : n.undetermined;

const scheduledNote = !granted
    ? n.deniedNote
    : status.count === 0
    ? n.scheduledNone
    : n.scheduled(
        status.count,
        status.until ? `${formatGregorianShort(status.until, t.date)}, ${formatHM(status.until)}` : '',
    );

const reminderLabel = (m: number) => (m === 0 ? n.reminderOff : n.reminderValue(m));

return (
    <Group label={n.group} note={scheduledNote}>
        <Row first>
            <Text style={settingsStyles.label}>{n.status}</Text>
            <Text style={[styles.value, !granted && styles.warn]}>{permissionText}</Text>
        </Row>
        {!granted && (
            <Pressable onPress={requestPermission} accessibilityRole="button" style={styles.linkRow}>
                <Text style={styles.link}>{n.allow}</Text>
            </Pressable>
        )}

        {granted && status.exact === false && (
            <>
                <Row>
                    <View style={styles.exactText}>
                        <View style={styles.exactHead}>
                            <Text style={settingsStyles.label}>{n.exact}</Text>
                            <Text style={[styles.value, styles.warn]}>{n.exactOff}</Text>
                        </View>
                        <Text style={styles.small}>{n.exactNote}</Text>
                    </View>
                </Row>
                <Pressable onPress={openExactAlarmSettings} accessibilityRole="button" style={styles.linkRow}>
                    <Text style={styles.link}>{n.exactAllow}</Text>
                </Pressable>
            </>
        )}

        {granted && (
            <>
                <NavRow label={n.reminder} value={reminderLabel(reminderMinutes)} onPress={() => setOpen((o) =>
!o)} />
                {open && (
                    <Choice
                        options={REMINDER_CHOICES.map((m) => ({ value: String(m), label: reminderLabel(m) })))
                        value={String(reminderMinutes)}

```

```

        onChange={() => {
            setReminderMinutes(Number(v));
            setOpen(false);
        }}
    />
    )}
    { /* на iPhone вибрацията се определя от настройките на телефона */}
    {Platform.OS === 'android' && <SwitchRow label={n.vibrate} value={vibrate} onChange={setVibrate}
/>}

    <NavRow label={n.diagnostics} onPress={() => router.push('/diagnostics')} />
  </>
  )}
</Group>
);
}

const styles = StyleSheet.create({
  text: { flex: 1, fontFamily: fonts.regular, fontSize: 13, lineHeight: 18, color: colors.textDim },
  value: { fontFamily: fonts.semibold, fontSize: 13.5, color: colors.textDim },
  warn: { color: colors.warn },
  small: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted },
  exactText: { flex: 1, gap: 4 },
  exactHead: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  linkRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
});

```

settings.tsx src/app/(tabs)

- Групата „Известия“ е веднага след „Място“.

src/app/(tabs)/settings.tsx

242 реда

```

import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useState } from 'react';
import { ActivityIndicator, Linking, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { NotificationsGroup } from '@components/settings/NotificationsGroup';
import { PinIcon } from '@components/icons';
import {
  Choice,
  Group,
  NavRow,
  Row,
  StepperRow,
  SwitchRow,
  settingsStyles,
} from '@components/settings/controls';
import { useTabBarHeight } from '@components/TabBar';
import { CALC_METHODS, HIGH_LAT_RULES, isHighLatitude, isPolar } from '@domain/calc';
import { PRAYER_IDS } from '@domain/prayers';
import { useI18n } from '@i18n';

```

```

import { refreshLocation, useLocationStatus } from '@services/location';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

export default function SettingsScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const s = useSettings();
  const location = selectLocation(s);
  const status = useLocationStatus((x) => x.status);
  const [open, setOpen] = useState<'method' | 'asr' | 'highLat' | null>(null);

  const isCalc = location.source === 'calc';
  const highLat = isCalc && isHighLatitude(location.latitude);
  const anyOffset = PRAYER_IDS.some((id) => s.offsets[id] !== 0);
  const toggle = (what: 'method' | 'asr' | 'highLat') => setOpen((cur) => (cur === what ? null : what));

  return (
    <View style={styles.root}>
      <GeometricPattern />
      /* списъкът свързва над долната навигация, за да не се застъпва с нея */
      <ScrollView
        style={{ marginBottom: tabBarHeight }}
        contentContainerStyle={{ styles.content, { paddingTop: insets.top + 8, paddingBottom: 24 } }}
        showsVerticalScrollIndicator={false}
      >
        <Text style={styles.title}>{t.settings.title}</Text>

        /* ---- Място ---- */
        <Group label={t.settings.groupPlace}>
          <Pressable
            onPress={() => router.push('/place')}
            accessibilityRole="button"
            accessibilityLabel={` ${location.names[lang]}. ${t.settings.change}`}
            style={({ pressed }) => [styles.placeRow, pressed && styles.pressed]}
          >
            <PinIcon size={18} color={colors.gold} />
            <View style={styles.placeText}>
              <Text style={styles.placeName}>{location.names[lang]}</Text>
              {location.detail ? <Text style={styles.placeDetail}>{location.detail[lang]}</Text> : null}
              <Text style={styles.badge}>{isCalc ? t.settings.sourceCalc : t.settings.sourceMufti}</Text>
            </View>
            <Text style={styles.link}>{t.settings.change}</Text>
          </Pressable>
          <SwitchRow label={t.settings.auto} value={s.autoLocation} onChange={s.setAutoLocation} />
          {s.autoLocation && (
            <Row>
              <View style={styles.statusBox}>
                <View style={styles.statusLine}>
                  {status === 'locating' && <ActivityIndicator size="small" color={colors.gold} />}
                  <Text style={[styles.statusText, status === 'locating' && styles.statusBusy]}>
                    {status === 'locating'
                      ? t.settings.locating
                      : status === 'denied'
                        ? t.settings.denied
                        : status === 'unavailable'
                          ? t.settings.unavailable
                    }
                  </Text>
                </View>
              </View>
            </Row>
          )}
        </Group>
      </ScrollView>
    </View>
  );
}

```

```

        : t.settings.autoHint}
      </Text>
    </View>
    {status === 'denied' ? (
      <Pressable onPress={() => Linking.openSettings()} accessibilityRole="button">
        <Text style={styles.link}>{t.settings.openSettings}</Text>
      </Pressable>
    ) : status !== 'locating' ? (
      <Pressable onPress={() => refreshLocation()} accessibilityRole="button">
        <Text style={styles.link}>{t.settings.refresh}</Text>
      </Pressable>
    ) : null}
  </View>
</Row>
)}
</Group>

{/* ---- Известия ---- */}
<NotificationsGroup />

{/* ---- Часове ---- */}
{isCalc ? (
  <Group
    label={t.settings.groupTimes}
    note={
      isPolar(location.latitude)
        ? t.settings.polarNote(Math.abs(location.latitude).toFixed(0))
        : highLat
        ? t.settings.highLatNote(Math.abs(location.latitude).toFixed(0))
        : t.settings.asrNote
    }
  >
    <NavRow first label={t.settings.method} value={t.methods[s.method]} onPress={() =>
toggle('method')} />
    {open === 'method' && (
      <Choice
        options={CALC_METHODS.map((m) => ({ value: m, label: t.methods[m] })))}
        value={s.method}
        onChange={(m) => {
          s.setMethod(m);
          setOpen(null);
        }}
      />
    )}
    <NavRow
      label={t.settings.asr}
      value={s.madhab === 'hanafi' ? t.settings.hanafi : t.settings.shafi}
      onPress={() => toggle('asr')}
    />
    {open === 'asr' && (
      <Choice
        options={[
          { value: 'shafi' as const, label: t.settings.shafi, desc: t.settings.shafiDesc },
          { value: 'hanafi' as const, label: t.settings.hanafi, desc: t.settings.hanafiDesc },
        ]}
        value={s.madhab}
        onChange={(m) => {
          s.setMadhab(m);
          setOpen(null);
        }}
      />
    )}
  )}

```

```

    })
    {highLat && (
        <>
        <NavRow
            label={t.settings.highLat}
            value={t.highLatRules[s.highLatRule].name}
            onPress={() => toggle('highLat')}
        />
        {open === 'highLat' && (
            <Choice
                options={HIGH_LAT_RULES.map((r) => ({
                    value: r,
                    label: t.highLatRules[r].name,
                    desc: t.highLatRules[r].desc,
                }))}
                value={s.highLatRule}
                onChange={(r) => {
                    s.setHighLatRule(r);
                    setOpen(null);
                }}
            />
        )}
    )}
</Group>
) : (
    <Group label={t.settings.groupTimes} note={t.settings.sourceMuftiNote}>
        <Row first>
            <Text style={settingsStyles.label}>{t.settings.sourceMufti}</Text>
        </Row>
    </Group>
)
})

/* ---- Коррекции ---- */
<Group label={t.settings.groupCorrections} note={t.settings.offsetsNote}>
    {PRAYER_IDS.map((id, i) => (
        <StepperRow
            key={id}
            first={i === 0}
            label={t.prayers[id]}
            valueText={t.settings.minutes(s.offsets[id])}
            highlighted={s.offsets[id] !== 0}
            onMinus={() => s.changeOffset(id, -1)}
            onPlus={() => s.changeOffset(id, 1)}
            minusLabel={t.settings.decrease(t.prayers[id])}
            plusLabel={t.settings.increase(t.prayers[id])}
        />
    ))}
    {anyOffset && (
        <Pressable onPress={s.resetOffsets} accessibilityRole="button" style={styles.resetRow}>
            <Text style={styles.link}>{t.settings.reset}</Text>
        </Pressable>
    )}
</Group>

<Group label={t.settings.hijri} note={isCalc ? t.settings.hijriNoteAbroad : t.settings.hijriNote}>
    <StepperRow
        first
        label={t.settings.hijri}
        valueText={t.settings.days(s.hijriAdjust)}
        highlighted={s.hijriAdjust !== 0}
    />

```

```

        onMinus={() => s.changeHijriAdjust(-1)}
        onPlus={() => s.changeHijriAdjust(1)}
        minusLabel={t.settings.decrease(t.settings.hijri)}
        plusLabel={t.settings.increase(t.settings.hijri)}
      />
    </Group>

    {/* ---- Приложение: източниците и поверителността са в „За приложението“ ---- */}
    <Group label={t.settings.groupApp}>
      <NavRow first label={t.settings.about} value={APP_VERSION} onPress={() => router.push('/about')} />
    </Group>
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { paddingHorizontal: 16, gap: 20 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginLeft: 4 },
  placeRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12 },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  placeText: { flex: 1, gap: 2 },
  placeName: { fontFamily: fonts.extrabold, fontSize: 17, color: colors.text },
  placeDetail: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.muted },
  badge: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.gold, marginTop: 2 },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
  statusBox: { flex: 1, gap: 6 },
  statusLine: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  statusText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  statusBusy: { color: colors.text },
  resetRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
});

```

TopBar.tsx src/components

- Първо пускане без място: докато GPS търси – „Определям мястото...“; ако не успее – „Избери място“, а не „София“.

src/components/TopBar.tsx

76 реда

```

import { ActivityIndicator, Pressable, StyleSheet, Text, View } from 'react-native';

import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { ChevronDownIcon, PinIcon } from './icons';

interface Props {
  city: string;
  gregorian: string;
  hijri: string;
}

```

```

onCityPress?: () => void;
/** GPS определя мястото в момента - въртящ се индикатор вместо иконата. */
locating?: boolean;
/**
 * Още няма място (първо отваряне, нито от GPS, нито избрано): докато GPS търси -
 * „Определям мястото...“, а ако не успее - „Избери място“, не „София“.
 */
locatingFirstTime?: boolean;
}

export function TopBar({ city, gregorian, hijri, onCityPress, locating, locatingFirstTime }: Props) {
  const { t } = useI18n();
  return (
    <View style={styles.bar}>
      <Pressable
        onPress={onCityPress}
        style={({ pressed }) => [styles.city, pressed && styles.pressed]}
        accessibilityRole="button"
        accessibilityLabel={locating ? `${t.a11y.locating}. ${t.a11y.city(city)}` : t.a11y.city(city)}
        accessibilityState={{ busy: !!locating }}
      >
        {locating ? (
          <ActivityIndicator size="small" color={colors.gold} style={styles.spinner} />
        ) : (
          <PinIcon color={colors.text} />
        )}
        <Text style={styles.cityText}>{locatingFirstTime ? (locating ? t.locating : t.choosePlace) : city}</Text>
        <ChevronDownIcon color={colors.text} />
      </Pressable>
      <View style={styles.dates}>
        <Text style={styles.gregorian}>{gregorian}</Text>
        <Text style={styles.hijri}>{hijri}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  bar: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 18,
    paddingTop: 8,
    paddingBottom: 4,
    gap: 8,
  },
  city: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 7,
    paddingLeft: 10,
    paddingRight: 12,
    borderRadius: 999,
    backgroundColor: colors.pill,
  },
  pressed: { opacity: 0.7 },
  spinner: { width: 15, height: 15, transform: [{ scale: 0.8 }] },
  cityText: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },

```

```

    dates: { alignItems: 'flex-end', flexShrink: 1 },
    gregorian: { fontFamily: fonts.bold, fontSize: 12, lineHeight: 16, color: colors.text },
    hijri: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.textDim },
  });

```

NoticeBanner.tsx src/components

- Без място (GPS не успя): „Мястото не е определено – часовете са за София. Докосни, за да избереш място.“ – води към избора на място.
- Без разрешение: „Известията не са разрешени – известията и алармите няма да работят. Докосни, за да ги разрешиш.“
- Android без „Аларми и напомнания“: „Известията може да закъснеят с няколко минути. Докосни...“.

src/components/NoticeBanner.tsx

82 реда

```

import { router } from 'expo-router';
import { Platform, Pressable, StyleSheet, Text } from 'react-native';

import { useI18n } from '@i18n';
import { useLocationStatus } from '@services/location';
import { openExactAlarmSettings, requestPermission, useNotificationStatus } from
  '@services/notifications';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

/**
 * Лента под горната лента на „Днес“, когато известията няма да дойдат навреме:
 * - известията са изключени → „Докосни, за да ги разрешиш“;
 * - Android без „Аларми и напомнания“ → известията може да закъснеят с минути.
 */
export function NoticeBanner() {
  const { t } = useI18n();
  const permission = useNotificationStatus((s) => s.permission);
  const exact = useNotificationStatus((s) => s.exact);
  const checked = useNotificationStatus((s) => s.checked);
  // още няма място (нищо от GPS, нито избрано), а GPS не успя – показват се часовете за София
  const noPlace = useSettings((s) => !s.gpsLocation && !s.manualLocation);
  const locationFailed = useLocationStatus((s) => s.status === 'denied' || s.status === 'unavailable');

  let text: string | null = null;
  let onPress: (() => void) | null = null;
  if (noPlace && locationFailed) {
    text = t.noPlaceBanner;
    onPress = () => router.push('/place');
  } else if (!checked) {
    return null;
  } else if (permission === 'denied' || permission === 'undetermined') {
    text = t.notifications.deniedBanner;
    onPress = requestPermission;
  } else if (permission === 'granted' && Platform.OS === 'android' && exact === false) {
    text = t.notifications.exactBanner;
    onPress = openExactAlarmSettings;
  }
  if (!text || !onPress) return null;

  return (
    <Pressable
      onPress={onPress}

```

```

    accessibilityRole="button"
    style={({ pressed }) => [styles.banner, pressed && styles.pressed]}
  >
    <Text style={styles.icon}>!/</Text>
    <Text style={styles.text}>{text}</Text>
  </Pressable>
);
}

const styles = StyleSheet.create({
  banner: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    marginHorizontal: 16,
    marginTop: 6,
    paddingHorizontal: 12,
    paddingVertical: 10,
    borderRadius: 14,
    backgroundColor: 'rgba(227,154,75,0.16)',
    borderWidth: 1,
    borderColor: 'rgba(227,154,75,0.45)',
  },
  pressed: { opacity: 0.75 },
  icon: {
    width: 22,
    height: 22,
    borderRadius: 11,
    textAlign: 'center',
    lineHeight: 22,
    fontFamily: fonts.extrabold,
    fontSize: 13,
    color: colors.base,
    backgroundColor: colors.warn,
    overflow: 'hidden',
    includeFontPadding: false,
  },
  text: { flex: 1, fontFamily: fonts.semibold, fontSize: 12.5, lineHeight: 17, color: colors.text },
});

```

index.tsx src/app/(tabs)

- Показва лентата. Без разрешение камбанката не се сменя: първо системният въпрос за разрешение (или настройките на телефона), **след него** съобщение „✓ Известията са разрешени“ или „Известията не са разрешени – известията и алармите няма да работят.“
- Съобщения на камбанките: „Иша: без известие“, „Иша: известие“, „Иша: аларма – звучи и при тих режим“. Думата е „аларма“, не „езан“ – в етап 6 звукът може да е и мелодия.

src/app/(tabs)/index.tsx

93 реда

```

import { router } from 'expo-router';
import { useCallback, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NextPrayerHero } from '@components/NextPrayerHero';
import { NoticeBanner } from '@components/NoticeBanner';
import { PhaseBackground } from '@components/PhaseBackground';
import { PrayerList } from '@components/PrayerList';

```

```

import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { TopBar } from '@components/TopBar';
import { formatGregorianShort } from '@domain/format';
import { formatHijri } from '@domain/hijri';
import type { PrayerId } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n } from '@i18n';
import { useLocationStatus } from '@services/location';
import { requestPermission, useNotificationStatus } from '@services/notifications';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, useSettings } from '@store/settings';

export default function TodayScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const tabBarHeight = useTabBarHeight();

  const now = useNow();
  const schedule = usePrayerSchedule(now);
  const location = useSettings(selectLocation);
  const hijriAdjust = useSettings((s) => s.hijriAdjust);
  const autoLocation = useSettings((s) => s.autoLocation);
  const hasPlace = useSettings((s) => s.gpsLocation !== null || s.manualLocation !== null);
  // „idle“ - само в първия миг преди първото търсене; броим го като търсене, за да не мигне „Избери място“
  const locating = useLocationStatus((s) => s.status === 'locating' || s.status === 'idle') &&
  autoLocation;

  const cycle = useAlertPrefs((s) => s.cycle);
  const notificationsOff = useNotificationStatus(
    (s) => s.checked && (s.permission === 'denied' || s.permission === 'undetermined'),
  );
  const [toast, setToast] = useState<{ id: number; text: string } | null>(null);

  const onBellPress = useCallback(
    (id: PrayerId) => {
      // Без разрешение камбанката не се сменя: първо питаме за разрешение (или отваряме
      // настройките на телефона). Съобщението се показва СЛЕД въпроса – иначе прозорецът
      // на системата го скрива и потребителят не успява да го прочете.
      if (notificationsOff) {
        requestPermission().then((p) =>
          setToast({
            id: Date.now(),
            text: p === 'granted' ? t.notifications.allowedToast : t.notifications.bellDenied,
          }),
        );
        return;
      }
      const mode = cycle(id);
      const text = mode === 'adhan' ? t.alarmToast(t.prayers[id]) : t.toast(t.prayers[id], t.alert[mode]);
      setToast({ id: Date.now(), text });
    },
    [cycle, t, notificationsOff],
  );

  return (
    <View style={styles.root}>
      <PhaseBackground phase={schedule.current.id} />

      <ScrollView

```

```

    contentContainerStyle={{ paddingTop: insets.top, paddingBottom: tabBarHeight + 16 }}
    showsVerticalScrollIndicator={false}
  >
    <TopBar
      city={location.names[lang]}
      gregorian={formatGregorianShort(now, t.date)}
      hijri={formatHijri(now, t.hijriMonths, hijriAdjust)}
      onCityPress={() => router.push('/place')}
      locating={locating}
      locatingFirstTime={!hasPlace}
    />
    <NoticeBanner />
    <NextPrayerHero schedule={schedule} />
    <PrayerList schedule={schedule} onBellPress={onBellPress} />
  </ScrollView>

  <Toast message={toast} bottom={tabBarHeight + 14} />
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
});

```

Toast.tsx src/components

- Съобщението стои според дължината на текста – от ~2 сек. до 6 сек., за да може да се прочете; дългите текстове се пренасят на два реда.

src/components/Toast.tsx

73 реда

```

import { useEffect } from 'react';
import { StyleSheet, Text } from 'react-native';
import Animated, {
  useAnimatedStyle,
  useSharedValue,
  withDelay,
  withSequence,
  withTiming,
} from 'react-native-reanimated';

import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

interface Props {
  /** Ново съобщение идва с нов `id`, за да се покаже отново и при същия текст. */
  message: { id: number; text: string } | null;
  bottom: number;
}

/**
 * Кратко съобщение над долната навигация. Стои според дължината на текста:
 * „Иша: езан“ – ~2 сек., дълго обяснение – до 6 сек., за да може да се прочете.
 */
export function toastDuration(text: string): number {
  return Math.min(6000, Math.max(1800, 1200 + text.length * 50));
}

export function Toast({ message, bottom }: Props) {

```

```

const progress = useSharedValue(0);

useEffect(() => {
  if (!message) return;
  progress.set(0);
  progress.set(
    withSequence(
      withTiming(1, { duration: 180 }),
      withDelay(toastDuration(message.text), withTiming(0, { duration: 220 })),
    ),
  );
}, [message, progress]);

const animatedStyle = useAnimatedStyle(() => ({
  opacity: progress.get(),
  transform: [{ translateY: (1 - progress.get()) * 8 }],
}));

if (!message) return null;

return (
  <Animated.View
    pointerEvents="none"
    accessibilityLiveRegion="polite"
    style={[styles.toast, { bottom }, animatedStyle]}
  >
    <Text style={styles.text}>{message.text}</Text>
  </Animated.View>
);
}

const styles = StyleSheet.create({
  toast: {
    position: 'absolute',
    alignSelf: 'center',
    maxWidth: '90%',
    paddingHorizontal: 16,
    paddingVertical: 10,
    borderRadius: 18,
    backgroundColor: 'rgba(8,12,22,0.92)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.12)',
  },
  text: { fontFamily: fonts.semibold, fontSize: 13, lineHeight: 18, color: colors.text, textAlign: 'center'
},
});

```

PrayerList.tsx src/components

- Без разрешение камбанките са бледи (`muted`) – веднага се вижда, че нищо няма да прозвучи.

src/components/PrayerList.tsx

48 реда

```

import { StyleSheet, View } from 'react-native';

import type { PrayerId, Schedule } from '@domain/prayers';
import { useNotificationStatus } from '@services/notifications';
import { useAlertPrefs } from '@store/alertPrefs';
import { colors } from '@theme/colors';

```

```
import { PrayerRow } from './PrayerRow';

interface Props {
  schedule: Schedule;
  onBellPress: (id: PrayerId) => void;
}

export function PrayerList({ schedule, onBellPress }: Props) {
  const modes = useAlertPrefs((s) => s.modes);
  // без разрешение за известия камбанките са бledi – нищо няма да прозвучи
  const muted = useNotificationStatus(
    (s) => s.checked && (s.permission === 'denied' || s.permission === 'undetermined'),
  );

  return (
    <View style={styles.card}>
      {schedule.today.map((p) => (
        <PrayerRow
          key={p.id}
          prayer={p}
          state={schedule.rowState(p.id)}
          mode={modes[p.id]}
          onBellPress={() => onBellPress(p.id)}
          muted={muted}
        />
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    marginHorizontal: 14,
    padding: 6,
    gap: 2,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
  },
});
```

BellButton.tsx src/components

- Нов параметър `muted` – прозрачност 30 %.

src/components/BellButton.tsx

47 реда

```
import { Pressable, StyleSheet } from 'react-native';

import type { AlertMode } from '@store/alertPrefs';
import { colors } from '@theme/colors';

import { AlertIcon } from './icons';

interface Props {
  mode: AlertMode;
  /** Готов текст за TalkBack/VoiceOver на текущия език. */
  accessibilityLabel: string;
  onPress: () => void;
}
```

```

/** Известията не са разрешени – камбанката е бледа: нищо няма да прозвучи. */
muted?: boolean;
}

/** Сиво = изключено, бяло = нотификация, злато = езан. */
export function BellButton({ mode, accessibilityLabel, onPress, muted }: Props) {
  const iconColor = mode === 'adhan' ? colors.goldInk : mode === 'notify' ? colors.text : colors.muted;

  return (
    <Pressable
      onPress={onPress}
      hitSlop={6}
      style={({ pressed }) => [styles.btn, styles[mode], muted && styles.muted, pressed && styles.pressed]}
      accessibilityRole="button"
      accessibilityLabel={accessibilityLabel}
    >
      <AlertIcon mode={mode} color={iconColor} />
    </Pressable>
  );
}

const styles = StyleSheet.create({
  btn: {
    width: 36,
    height: 36,
    borderRadius: 18,
    alignItems: 'center',
    justifyContent: 'center',
  },
  off: { backgroundColor: 'transparent' },
  notify: { backgroundColor: colors.bellNotify },
  adhan: { backgroundColor: colors.gold },
  muted: { opacity: 0.3 },
  pressed: { transform: [{ scale: 0.92 }] },
});

```

diagnostics.tsx src/app

- **Състояние** с отметки: разрешение, точни известия, колко са планирани. При проблем редът е оранжев с бутон „Разреша“.
- **Пробно известие**: два бутона – „Известие“ и „Аларма“; след натискане бутонът брои до известието.
- **Последни известия**: кога е планирано, кога е показано и знак „навреме“ (зелено) или „+4,2 мин.“ (оранжево).
- **Следващи известия** с час, ден и вид (Аларма / Известие / Напомняне).
- **Технически данни** – сгънати: телефон, версия на Android, каналите.
- „Обнови“ само четете наново от телефона (индикатор и „Обновено в ...“) – нищо не променя.
- „Пренасрочи всички известия“ трие всички известия на Езан и ги планира наново по текущите настройки – ако нещо изглежда объркано.

src/app/diagnostics.tsx

489 реда

```

import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { ActivityIndicator, Platform, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

```

```

import { GeometricPattern } from '@components/GeometricPattern';
import { AlertIcon, ChevronDownIcon, ChevronIcon } from '@components/icons';
import { formatGregorianShort, formatHM } from '@domain/format';
import type { SoundKind } from '@domain/notifications';
import { useI18n } from '@i18n';
import {
  openExactAlarmSettings,
  refreshPermission,
  requestPermission,
  resetAllNotifications,
  sendTestNotification,
  useNotificationStatus,
} from '@services/notifications';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

/* ----- данни */

type Kind = 'alarm' | 'notification' | 'reminder' | 'other';

interface Snapshot {
  /** Кога са прочетени данните - показва се под заглавието. */
  loadedAt: Date;
  channels: Notifications.NotificationChannel[];
  upcoming: { id: string; at: Date; title: string; kind: Kind }[];
  recent: { id: string; planned: Date | null; shown: Date; title: string }[];
}

const pad = (n: number) => String(n).padStart(2, '0');
const hms = (d: Date) => `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`;
const USAGE: Record<number, string> = { 4: 'alarm', 5: 'notification' };

const atOf = (data: unknown) => {
  const at = (data as { at?: number } | null)?.at;
  return typeof at === 'number' ? new Date(at) : null;
};

/** Видът на известието по канала му (Android) или по данните (iPhone). */
function kindOf(r: Notifications.NotificationRequest): Kind {
  const channel = (r.trigger as { channelId?: string } | null)?.channelId ?? '';
  const data = r.content.data as { kind?: string; sound?: string } | null;
  if (channel.startsWith('adhan') || data?.sound === 'adhan') return 'alarm';
  if (channel.startsWith('reminder') || data?.kind === 'reminder' || data?.kind === 'refresh') return 'reminder';
  if (channel.startsWith('prayer') || data?.kind === 'prayer') return 'notification';
  return 'other';
}

async function fetchSnapshot(): Promise<Snapshot> {
  if (Platform.OS === 'web') return { loadedAt: new Date(), channels: [], upcoming: [], recent: [] };
  await refreshPermission();
  const [channels, scheduled, presented] = await Promise.all([
    Platform.OS === 'android' ? Notifications.getNotificationChannelsAsync() : Promise.resolve([]),
    Notifications.getAllScheduledNotificationsAsync(),
    Notifications.getPresentedNotificationsAsync(),
  ]);
  return {
    loadedAt: new Date(),
    channels: channels.filter((c) => !c.id.startsWith('expo_')),
    upcoming: scheduled
  };
}

```

```

        .map((r) => ({ id: r.identifier, at: atOf(r.content.data) ?? new Date(0), title: r.content.title ??
'', kind: kindOf(r) })))
        .sort((a, b) => a.at.getTime() - b.at.getTime()),
    recent: presented
    .map((n) => ({
        id: n.request.identifier,
        planned: atOf(n.request.content.data),
        shown: new Date(n.date),
        title: n.request.content.title ?? '',
    })))
    .sort((a, b) => b.shown.getTime() - a.shown.getTime()),
};
}

/* ----- екран */

/**
 * „Проверка на известията“ (Настройки → Известия):
 * състояние с отметки, пробно известие, последните показани (със закъснението),
 * следващите планирани и – сгънати – техническите данни.
 */
export default function DiagnosticsScreen() {
    const insets = useSafeAreaInsets();
    const { t } = useI18n();
    const d = t.notifications.diag;
    const status = useNotificationStatus();
    const [snap, setSnap] = useState<Snapshot | null>(null);
    const [busy, setBusy] = useState(false);
    const [techOpen, setTechOpen] = useState(false);
    const [test, setTest] = useState<{ sound: SoundKind; at: number } | null>(null);
    const [now, setNow] = useState(() => Date.now());

    const [refreshing, setRefreshing] = useState(false);
    const [resetDone, setResetDone] = useState(false);
    const load = useCallback(() => fetchSnapshot().then(setSnap), []);

    // „Обнови“: чете наново от телефона; индикаторът стои поне 0,6 сек., за да се забележи
    const refresh = async () => {
        setRefreshing(true);
        await Promise.all([load(), new Promise((r) => setTimeout(r, 600))]);
        setRefreshing(false);
    };

    useEffect(() => {
        let alive = true;
        fetchSnapshot().then((x) => alive && setSnap(x));
        return () => {
            alive = false;
        };
    }, []);

    // обратно броене на пробното известие; след него – обновяване, за да се види в „Последни“
    useEffect(() => {
        if (!test) return;
        const timer = setInterval(() => {
            const t2 = Date.now();
            setNow(t2);
            if (t2 >= test.at + 1500) {
                setTest(null);
                load();
            }
        }
    });
}

```

```

    }, 500);
    return () => clearInterval(timer);
  }, [test, load]);

const closing = useRef(false);
const back = () => {
  if (closing.current) return;
  closing.current = true;
  if (router.canGoBack()) router.back();
  else router.replace('/settings');
};

const sendTest = async (sound: SoundKind) => {
  const at = await sendTestNotification(sound);
  setNow(at - 10_000);
  setTest({ sound, at });
};

const reset = async () => {
  setBusy(true);
  setResetDone(false);
  await resetAllNotifications();
  await load();
  setBusy(false);
  setResetDone(true);
};

const granted = status.permission === 'granted';
const kindLabel: Record<Kind, string> = {
  alarm: d.kindAlarm,
  notification: d.kindNotification,
  reminder: d.kindReminder,
  other: d.kindOther,
};

const until = status.until ? `${formatGregorianShort(status.until, t.date)}, ${formatHM(status.until)}` :
'';

const device =
  Platform.OS === 'android'
    ? `${Platform.constants.Manufacturer} ${Platform.constants.Model} · Android
    ${Platform.constants.Release} (API ${Platform.Version})`
    : `${Platform.OS} ${Platform.Version}`;

const testButton = (sound: SoundKind, label: string) => {
  const active = test?.sound === sound;
  const left = active ? Math.max(0, Math.ceil((test.at - now) / 1000)) : 0;
  return (
    <Pressable
      onPress={() => sendTest(sound)}
      disabled={!test || !granted}
      accessibilityRole="button"
      style={({ pressed }) => [
        styles.testBtn,
        sound === 'adhan' && styles.testBtnGold,
        (!test && !active) || !granted ? styles.disabled : null,
        pressed && styles.pressed,
      ]}
    >
      <AlertIcon mode={sound === 'adhan' ? 'adhan' : 'notify'} size={20} color={sound === 'adhan' ?
colors.goldInk : colors.text} />
      <Text style={[styles.testLabel, sound === 'adhan' && styles.testLabelGold]}>{label}</Text>
      {active && <Text style={[styles.testIn, sound === 'adhan' && styles.testLabelGold]}>

```

```

{d.testIn(left)}</Text>
    </Pressable>
  );
};

return (
  <View style={styles.root}>
    <GeometricPattern />
    <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
      <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
        <ChevronIcon direction="left" size={18} color={colors.gold} />
        <Text style={styles.link}>{t.settings.title}</Text>
      </Pressable>
      <Pressable onPress={refresh} disabled={refreshing} hitSlop={10} accessibilityRole="button" style=
{styles.refreshBtn}>
        {refreshing ? <ActivityIndicator size="small" color={colors.gold} /> : <Text style={styles.link}>
{d.refresh}</Text>}
      </Pressable>
    </View>

    <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}>
      <View style={styles.titleBox}>
        <Text style={styles.title}>{d.title}</Text>
        <Text style={styles.subtitle}>{d.subtitle}</Text>
        {snap && <Text style={styles.updated}>{d.updated(hms(snap.loadedAt))}</Text>}
      </View>

      {/* ---- Состояние ---- */}
      <Section label={d.status}>
        <Check
          ok={granted}
          text={granted ? d.permOk : d.permBad}
          action={granted ? undefined : { label: d.allow, onPress: requestPermission }}
          first
        />
        {status.exact !== null && (
          <Check
            ok={status.exact}
            text={status.exact ? d.exactOk : d.exactBad}
            action={status.exact ? undefined : { label: d.allow, onPress: openExactAlarmSettings }}
          />
        )}
        <Check ok={status.count > 0} text={status.count > 0 ? d.scheduledOk(status.count, until) :
d.scheduledNone} />
      </Section>

      {/* ---- Пробно известие ---- */}
      <Section label={d.test}>
        <View style={styles.testBox}>
          <View style={styles.testRow}>
            {testButton('chime', d.testNotification)}
            {testButton('adhan', d.testAlarm)}
          </View>
          <Text style={styles.small}>{d.testHint}</Text>
        </View>
      </Section>

      {!snap ? (
        <ActivityIndicator color={colors.gold} style={{ marginTop: 12 }} />
      ) : (
        <>

```

```

    { /* ---- Последни ---- */
    <Section label={d.recent}>
      {snap.recent.length === 0 ? (
        <Line first>
          <Text style={styles.small}>{d.recentNone}</Text>
        </Line>
      ) : (
        snap.recent.slice(0, 8).map((p, i) => {
          const delay = p.planned ? Math.round((p.shown.getTime() - p.planned.getTime()) / 1000) :
null;

          const onTime = delay !== null && delay < 60;
          return (
            <Line key={p.id + i} first={i === 0}>
              <View style={styles.item}>
                <Text style={styles.itemTitle} numberOfLines={1}>
                  {p.title}
                </Text>
                <Text style={styles.small}>
                  {p.planned ? d.plannedShown(hms(p.planned), hms(p.shown)) : hms(p.shown)}
                </Text>
              </View>
              {delay !== null && (
                <View style={[styles.badge, onTime ? styles.badgeOk : styles.badgeWarn]}>
                  <Text style={[styles.badgeText, onTime ? styles.badgeTextOk :
styles.badgeTextWarn]}>
                    {onTime ? d.onTime : d.late(delay)}
                  </Text>
                </View>
              )}
            </Line>
          );
        })
      )}
    </Section>

    { /* ---- Следващи ---- */
    {snap.upcoming.length > 0 && (
      <Section label={d.upcoming}>
        {snap.upcoming.slice(0, 8).map((r, i) => (
          <Line key={r.id} first={i === 0}>
            <View style={styles.timeCol}>
              <Text style={styles.time}>{formatHM(r.at)}</Text>
              <Text style={styles.tiny}>{t.date.weekdaysShort[r.at.getDay()]}</Text>
            </View>
            <Text style={[styles.itemTitle, styles.flex]} numberOfLines={1}>
              {r.title}
            </Text>
            <View style={[styles.tag, r.kind === 'alarm' && styles.tagGold]}>
              <Text style={[styles.tagText, r.kind === 'alarm' && styles.tagTextGold]}>
{kindLabel[r.kind]}</Text>
            </View>
          </Line>
        )})
      </Section>
    )}

    { /* ---- Технически данни (сгънати) ---- */
    <View style={styles.card}>
      <Pressable
        onPress={() => setTechOpen((o) => !o)}
        accessibilityRole="button"

```

```

        accessibilityState={{ expanded: techOpen }}
        style={styles.techHead}
      >
        <Text style={styles.itemTitle}>{d.tech}</Text>
        <View style={techOpen && styles.flip}>
          <ChevronDownIcon color={colors.muted} />
        </View>
      </Pressable>
      {techOpen && (
        <View style={styles.techBody}>
          <Text style={styles.small}>
            {d.device}: {device}
          </Text>
          {snap.channels.map((c) => (
            <Text key={c.id} style={styles.small}>
              <Text style={styles.mono}>{c.id}</Text> -{' '}
              {d.channelLine(
                c.importance,
                c.sound ?? '-',
                c.enableVibrate,
                USAGE[c.audioAttributes?.usage ?? -1] ?? String(c.audioAttributes?.usage ?? '-'),
              )}
            </Text>
          ))}
        </View>
      )}
    </View>

    <View style={styles.resetBox}>
      <Pressable onPress={reset} disabled={busy} accessibilityRole="button" style=
{styles.resetBtn}>
        {busy ? <ActivityIndicator color={colors.gold} /> : <Text style={styles.link}>{d.reset}
      </Text>}
      </Pressable>
      <Text style={[styles.small, styles.center]}>{resetDone ? d.resetDone : d.resetHint}</Text>
    </View>
  </>
)
</ScrollView>
</View>
);
}

/* ----- ЧАСТИ */

function Section({ label, children }: { label: string; children: React.ReactNode }) {
  return (
    <View style={styles.section}>
      <Text style={styles.sectionLabel}>{label.toUpperCase()}</Text>
      <View style={styles.card}>{children}</View>
    </View>
  );
}

function Line({ children, first }: { children: React.ReactNode; first?: boolean }) {
  return <View style={[styles.line, !first && styles.lineBorder]}>{children}</View>;
}

function Check({
  ok,
  text,

```

```

    action,
    first,
  }: {
    ok: boolean;
    text: string;
    action?: { label: string; onPress: () => void };
    first?: boolean;
  }) {
    return (
      <Line first={first}>
        <View style={[styles.dot, ok ? styles.dotOk : styles.dotWarn]}>
          <Text style={styles.dotText}>{ok ? '✓' : '!'}</Text>
        </View>
        <Text style={[styles.checkText, !ok && styles.checkWarn]}>{text}</Text>
        {action && (
          <Pressable onPress={action.onPress} hitSlop={8} accessibilityRole="button" style=
{styles.actionBtn}>
            <Text style={styles.actionText}>{action.label}</Text>
          </Pressable>
        )}
      </Line>
    );
  }

const GREEN = '#5FBF8F';

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: {
    paddingHorizontal: 16,
    paddingBottom: 4,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6 },
  link: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 20 },
  titleBox: { gap: 6, marginTop: 4 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
  subtitle: { fontFamily: fonts.regular, fontSize: 13, lineHeight: 19, color: colors.muted },

  section: { gap: 7 },
  sectionLabel: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted,
paddingLeft: 6 },
  card: {
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.06)',
    borderRadius: 18,
    overflow: 'hidden',
  },
  line: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12,
minHeight: 52 },
  lineBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },

  dot: { width: 24, height: 24, borderRadius: 12, alignItems: 'center', justifyContent: 'center' },
  dotOk: { backgroundColor: 'rgba(95,191,143,0.18)' },
  dotWarn: { backgroundColor: colors.warn },
  dotText: { fontFamily: fonts.extrabold, fontSize: 13, color: colors.text, includeFontPadding: false },
  checkText: { flex: 1, fontFamily: fonts.semibold, fontSize: 14, lineHeight: 19, color: colors.text },

```

```

checkWarn: { color: colors.warn },
actionBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999, backgroundColor: colors.gold
},
actionText: { fontFamily: fonts.bold, fontSize: 12.5, color: colors.goldInk },

testBox: { padding: 14, gap: 12 },
testRow: { flexDirection: 'row', gap: 10 },
testBtn: {
  flex: 1,
  height: 78,
  borderRadius: 16,
  alignItems: 'center',
  justifyContent: 'center',
  gap: 4,
  backgroundColor: 'rgba(255,255,255,0.08)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.10)',
},
testBtnGold: { backgroundColor: colors.gold, borderColor: colors.gold },
testLabel: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
testLabelGold: { color: colors.goldInk },
testIn: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.textDim, ...tabularNums },
disabled: { opacity: 0.4 },
pressed: { transform: [{ scale: 0.97 }] },

item: { flex: 1, gap: 2 },
flex: { flex: 1 },
itemTitle: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
small: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted },
tiny: { fontFamily: fonts.medium, fontSize: 10.5, color: colors.muted },
mono: { fontFamily: fonts.bold, color: colors.textDim },

badge: { paddingHorizontal: 9, paddingVertical: 4, borderRadius: 999 },
badgeOk: { backgroundColor: 'rgba(95,191,143,0.16)' },
badgeWarn: { backgroundColor: 'rgba(227,154,75,0.18)' },
badgeText: { fontFamily: fonts.bold, fontSize: 11.5, ...tabularNums },
badgeTextOk: { color: GREEN },
badgeTextWarn: { color: colors.warn },

timeCol: { width: 46, alignItems: 'flex-start' },
time: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.text, ...tabularNums },
tag: { paddingHorizontal: 8, paddingVertical: 3, borderRadius: 999, backgroundColor:
'rgba(255,255,255,0.08)' },
tagGold: { backgroundColor: 'rgba(212,168,87,0.18)' },
tagText: { fontFamily: fonts.semibold, fontSize: 11, color: colors.textDim },
tagTextGold: { color: colors.gold },

techHead: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  paddingHorizontal: 14,
  paddingVertical: 14,
},
flip: { transform: [{ rotate: '180deg' }] },
techBody: {
  paddingHorizontal: 14,
  paddingBottom: 14,
  gap: 6,
  borderTopWidth: StyleSheet.hairlineWidth,
  borderTopColor: 'rgba(255,255,255,0.10)',

```

```

paddingTop: 10,
},
resetBox: { gap: 6, alignItems: 'center' },
resetBtn: {
  alignSelf: 'stretch',
  alignItems: 'center',
  paddingVertical: 13,
  borderRadius: 14,
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.45)',
},
center: { textAlign: 'center', paddingHorizontal: 12 },
refreshBtn: { minWidth: 70, alignItems: 'flex-end' },
updated: { fontFamily: fonts.medium, fontSize: 11.5, color: colors.muted, ...tabularNums },
});

```

controls.tsx src/components/settings

- Изборът с радио бутони: текстът е без `flex: 1` и без допълнителния отстъп на шрифта в Android (`includeFontPadding: false`) – вече е на една линия с бутона.

src/components/settings/controls.tsx

260 реда

```

import type { ReactNode } from 'react';
import { Pressable, StyleSheet, Switch, Text, View } from 'react-native';

import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

import { ChevronIcon } from '../icons';

/* Градивните елементи на екрана „Настройки“ – по тоскур-а: група с етикет и полупрозрачна карта. */

export function Group({ label, children, note }: { label: string; children: ReactNode; note?: string }) {
  return (
    <View style={styles.group}>
      <Text style={styles.groupLabel}>{label.toUpperCase()}</Text>
      <View style={styles.card}>{children}</View>
      {note ? <Text style={styles.note}>{note}</Text> : null}
    </View>
  );
}

export function Row({ children, first }: { children: ReactNode; first?: boolean }) {
  return <View style={[styles.row, !first && styles.rowBorder]}>{children}</View>;
}

export function NavRow({
  label,
  value,
  onPress,
  first,
}: {
  label: string;
  value?: string;
  onPress: () => void;
  first?: boolean;
}) {
  return (
    <Pressable

```

```

    onPress={onPress}
    accessibilityRole="button"
    style={({ pressed }) => [styles.row, !first && styles.rowBorder, pressed && styles.pressed]}
  >
    <Text style={styles.label}>{label}</Text>
    {value ? (
      <Text style={styles.value} numberOfLines={1}>
        {value}
      </Text>
    ) : null}
    <ChevronIcon direction="right" size={16} color={colors.muted} />
  </Pressable>
);
}

export function SwitchRow({
  label,
  value,
  onChange,
  first,
}: {
  label: string;
  value: boolean;
  onChange: (v: boolean) => void;
  first?: boolean;
}) {
  return (
    <Row first={first}>
      <Text style={styles.label}>{label}</Text>
      <Switch
        value={value}
        onValueChange={onChange}
        trackColor={{ false: 'rgba(255,255,255,0.18)', true: colors.gold }}
        thumbColor="#FFFFFF"
        accessibilityLabel={label}
      />
    </Row>
  );
}

export function Segmented<T extends string>({
  options,
  value,
  onChange,
}: {
  options: { value: T; label: string }[];
  value: T;
  onChange: (v: T) => void;
}) {
  return (
    <View style={styles.seg} accessibilityRole="radiogroup">
      {options.map((o) => {
        const on = o.value === value;
        return (
          <Pressable
            key={o.value}
            onPress={() => onChange(o.value)}
            accessibilityRole="radio"
            accessibilityState={{ checked: on }}
            style={[styles.segBtn, on && styles.segOn]}
          >

```

```

        <Text style={[styles.segText, on && styles.segTextOn]}>{o.label}</Text>
      </Pressable>
    );
  }
}
</View>
);
}

/** Ред с „-“ стойност +“. */
export function StepperRow({
  label,
  valueText,
  onMinus,
  onPlus,
  minusLabel,
  plusLabel,
  highlighted,
  first,
}: {
  label: string;
  valueText: string;
  onMinus: () => void;
  onPlus: () => void;
  minusLabel: string;
  plusLabel: string;
  highlighted?: boolean;
  first?: boolean;
}) {
  return (
    <Row first={first}>
      <Text style={styles.label}>{label}</Text>
      <View style={styles.stepper}>
        <Pressable onPress={onMinus} hitSlop={6} style={styles.stepBtn} accessibilityRole="button"
accessibilityLabel={minusLabel}>
          <Text style={styles.stepSign}>-</Text>
        </Pressable>
        <Text style={[styles.stepValue, highlighted && styles.gold]}>{valueText}</Text>
        <Pressable onPress={onPlus} hitSlop={6} style={styles.stepBtn} accessibilityRole="button"
accessibilityLabel={plusLabel}>
          <Text style={styles.stepSign}>+</Text>
        </Pressable>
      </View>
    </Row>
  );
}

/** Списък с избор на една стойност (радио бутони) – за метод и правило за северни ширини. */
export function Choice<T extends string>({
  options,
  value,
  onChange,
}: {
  options: { value: T; label: string; desc?: string }[];
  value: T;
  onChange: (v: T) => void;
}) {
  return (
    <View accessibilityRole="radiogroup">
      {options.map((o, i) => {
        const on = o.value === value;
        return (

```

```

        <Pressable
          key={o.value}
          onPress={() => onChange(o.value)}
          accessibilityRole="radio"
          accessibilityState={{ checked: on }}
          style={({ pressed }) => [styles.row, i > 0 && styles.rowBorder, pressed && styles.pressed]}
        >
          <View style={[styles.radio, on && styles.radioOn]}>{on && <View style={styles.radioDot} />}
        </View>

          <View style={styles.choiceText}>
            <Text style={[styles.choiceLabel, on && styles.labelOn]}>{o.label}</Text>
            {o.desc ? <Text style={styles.desc}>{o.desc}</Text> : null}
          </View>
        </Pressable>
      );
    }
  </View>
);
}

export const settingsStyles = StyleSheet.create({
  label: { flex: 1, fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
  value: { fontFamily: fonts.medium, fontSize: 13.5, color: colors.muted, maxWidth: '55%' },
});

const styles = StyleSheet.create({
  group: { gap: 7 },
  groupLabel: {
    fontFamily: fonts.bold,
    fontSize: 11,
    letterSpacing: 1.1,
    color: colors.muted,
    paddingLeft: 6,
  },
  card: {
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.06)',
    borderRadius: 18,
    overflow: 'hidden',
  },
  note: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted, paddingHorizontal:
6 },
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    minHeight: 50,
    paddingHorizontal: 14,
    paddingVertical: 8,
  },
  rowBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  label: settingsStyles.label,
  labelOn: { color: colors.gold },
  value: settingsStyles.value,
  desc: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted, marginTop: 1 },
  seg: { flexDirection: 'row', padding: 3, borderRadius: 999, backgroundColor: 'rgba(255,255,255,0.08)' },
  segBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999 },
  segOn: { backgroundColor: colors.text },
  segText: { fontFamily: fonts.bold, fontSize: 13, color: colors.muted },

```

```

segTextOn: { color: colors.base },
stepper: { flexDirection: 'row', alignItems: 'center', gap: 4 },
stepBtn: {
  width: 32,
  height: 32,
  borderRadius: 16,
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.16)',
  alignItems: 'center',
  justifyContent: 'center',
},
stepSign: { fontFamily: fonts.semibold, fontSize: 18, lineHeight: 20, color: colors.text },
stepValue: {
  minWidth: 62,
  textAlign: 'center',
  fontFamily: fonts.bold,
  fontSize: 13.5,
  color: colors.text,
  ...tabularNums,
},
gold: { color: colors.gold },
radio: {
  width: 20,
  height: 20,
  borderRadius: 10,
  borderWidth: 2,
  borderColor: 'rgba(255,255,255,0.3)',
  alignItems: 'center',
  justifyContent: 'center',
},
radioOn: { borderColor: colors.gold },
radioDot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.gold },
// Без flex:1 и без допълнителния отстъп на шрифта в Android – иначе текстът стои
// по-високо от радио бутона.
choiceText: { flex: 1, justifyContent: 'center' },
choiceLabel: {
  fontFamily: fonts.semibold,
  fontSize: 14,
  lineHeight: 20,
  color: colors.text,
  includeFontPadding: false,
  textAlignVertical: 'center',
},
});

```

Стъпка 8

Поправки след първата проба на телефона

Четири неща от първата проба и какво е направено.

ПРОБЛЕМ	ПРИЧИНА	ПОПРАВКА
Текстът в „Напомняне преди молитвата“ е по-високо от радио бутона	flex: 1 на текста и допълнителният отстъп на шрифта в Android	поправено в controls.tsx
Напомнянето (5 мин.) дойде 1 мин. преди молитвата	най-вероятно Android не позволява точни известия („Аларми и напомняния“) или пестенето на батерия ги отлага	лента на „Днес“ с бутон към настройката; „Проверка на известията“ показва закъснението; напълно надеждното решение е алармата на етап 5 (setAlarmClock)
Езанът не прозвуча в часа на Магриб	звукът на известие зависи от силата за известия, безшумния режим и от „изчакването между известия“ на Android 15+ (напомнянето дойде минута преди това)	каналът „Езан“ звучи през потока за аларми (версия 2 на каналите)
Вибрацията е включена, но не вибрира	за проверка – в „Проверка на известията“ се вижда дали каналът е с вибрация; на някои телефони вибрацията за известия е изключена в настройките на системата	нов канал с вибрация 3 × 500 мс

Каналите от по-старите версии се трият автоматично при първото пускане на новата версия, а известията се пренасрочват в новите канали.

Втора проба (Samsung A54, Android 16): известията и езанът дойдоха навреме. Поправено още: вибрацията при езан продължава, докато звучи езанът (при известие – една); по-подробен текст със следващата молитва; без разрешение камбанките са бледи и не се сменят.

Стъпка 9

Тестове

112 теста общо, 12 от тях нови.

notifications.test.ts src/domain/__tests__

- Часовете са точно тези от „Днес“, по ред, само бъдещите; текстовете на български.
- Изгрев – винаги без езан; изключена камбанка – без известие; всички изключени – празен план.
- Напомняне 15 мин. – с кратък звук, точно преди самото известие.
- Ограничението 62 и „Отвори Езан“ накрая; без напомнания 7-те дни се събират.
- Известие след по-малко от 5 сек. не се планира; отпечатъкът се сменя при друга настройка; полярна нощ – всички дати са валидни.

src/domain/__tests__/notifications.test.ts

121 реда

```
import { bg } from '@i18n/strings';

import { NOTIFICATION_LIMIT, planNotifications, planSignature, type AlertMode, type PlanInput } from
'../notifications';
import type { PrayerId } from '../prayers';
import { locationFromCoords } from '../resolve';
import { computeDay } from '../times';

const chepintsi = locationFromCoords(41.4352, 24.869);
const options = { location: chepintsi, method: 'Turkey' as const, madhab: 'shafi' as const };
const texts = { prayers: bg.prayers, ...bg.notifications };

const ALL: Record<PrayerId, AlertMode> = {
  fajr: 'adhan',
  sunrise: 'notify',
  dhuhr: 'notify',
  asr: 'adhan',
  maghrib: 'adhan',
  isha: 'notify',
};

const input = (over: Partial<PlanInput> = {}): PlanInput => ({
  now: new Date(2026, 8, 30, 12, 0), // 30.09.2026, 12:00 – преди Зухр (13:16)
  options,
  modes: ALL,
  reminderMinutes: 0,
  placeName: 'Чепинци',
  texts,
  ...over,
});

describe('план на известията', () => {
  it('часовете са точно тези от „Днес“, по ред, само бъдещите', () => {
    const plan = planNotifications(input());
    const today = computeDay(new Date(2026, 8, 30, 12), options);
    // днес остават Зухр, Аср, Магриб, Иша
    expect(plan.slice(0, 4).map((n) => n.prayer)).toEqual(['dhuhr', 'asr', 'maghrib', 'isha']);
    expect(plan[0].at.getTime()).toBe(today.find((p) => p.id === 'dhuhr')!.time.getTime());
    for (let i = 1; i < plan.length; i++) expect(plan[i].at.getTime()).toBeGreaterThan(plan[i -
1].at.getTime());
  });

  it('текстовете: „Магриб – 19:13“ + следващата молитва', () => {
```

```

const maghrib = planNotifications(input()).find((n) => n.prayer === 'maghrib')!;
expect(maghrib.title).toBe('Магриб - 19:13');
expect(maghrib.body).toBe('Време е за намаза Магриб (المغرب) · Чепинци\nСледваща: Иша в 20:35');
expect(maghrib.sound).toBe('adhan');
expect(maghrib.id).toBe('ezan-20260930-maghrib-prayer');
});

it('изгревът е винаги само известие (без езан), с текст за края на Фаджр', () => {
  const plan = planNotifications(input({ modes: { ...ALL, sunrise: 'adhan' } }));
  const sunrise = plan.find((n) => n.prayer === 'sunrise')!;
  expect(sunrise.sound).toBe('chime');
  expect(sunrise.body).toBe('Изгрев - края на времето за Фаджр · Чепинци\nСледваща: Зухр в 13:16');
});

it('след Иша следващата е Фаджр „(утре)“ - с часа от следващия ден', () => {
  const isha = planNotifications(input()).find((n) => n.id === 'ezan-20260930-isha-prayer')!;
  const fajr = computeDay(new Date(2026, 9, 1, 12), options)[0].time;
  const hm = `${String(fajr.getHours()).padStart(2, '0')}:${String(fajr.getMinutes()).padStart(2, '0')}`;
  expect(isha.body.split('\n')[1]).toBe(`Следваща: Фаджр в ${hm} (утре)`);
});

it('изключена камбанка - няма известие за тази молитва', () => {
  const plan = planNotifications(input({ modes: { ...ALL, isha: 'off' } }));
  expect(plan.some((n) => n.prayer === 'isha')).toBe(false);
});

it('всички изключени - празен план', () => {
  const off = Object.fromEntries(Object.keys(ALL).map((k) => [k, 'off'])) as Record<PrayerId, AlertMode>;
  expect(planNotifications(input({ modes: off }))).toEqual([]);
});

it('напомняне 15 мин. преди - с кратък звук, преди самото известие', () => {
  const plan = planNotifications(input({ reminderMinutes: 15 }));
  const i = plan.findIndex((n) => n.id === 'ezan-20260930-maghrib-reminder');
  expect(plan[i].title).toBe('Магриб след 15 мин. - 19:13');
  expect(plan[i].body).toBe('Време е за абдест и подготовка за намаза · Чепинци');
  expect(plan[i].sound).toBe('chime');
  expect(plan[i + 1].id).toBe('ezan-20260930-maghrib-prayer');
  expect(plan[i + 1].at.getTime() - plan[i].at.getTime()).toBe(15 * 60_000);
});

it('до 62 известия (iOS пази до 64); последното е „Отвори Езан“', () => {
  const plan = planNotifications(input({ reminderMinutes: 10 }));
  expect(plan).toHaveLength(NOTIFICATION_LIMIT);
  const last = plan[plan.length - 1];
  expect(last.kind).toBe('refresh');
  expect(last.at.getTime()).toBeGreaterThan(plan[plan.length - 2].at.getTime());
});

it('без напомняния 7 дни се събират без „Отвори Езан“', () => {
  const plan = planNotifications(input());
  expect(plan.length).toBeLessThanOrEqual(NOTIFICATION_LIMIT);
  expect(plan.some((n) => n.kind === 'refresh')).toBe(false);
  expect(plan.filter((n) => n.prayer === 'maghrib')).toHaveLength(7);
});

it('известие след по-малко от 5 сек. не се планира', () => {
  const today = computeDay(new Date(2026, 8, 30, 12), options);
  const dhuhr = today.find((p) => p.id === 'dhuhr')!.time;
  const plan = planNotifications(input({ now: new Date(dhuhr.getTime() - 2_000) }));
  expect(plan[0].prayer).not.toBe('dhuhr');
}

```

```
});

it('отпечатъкът се сменя при друга настройка и е еднакъв при еднакъв план', () => {
  const a = planSignature(planNotifications(input()), 'v');
  const b = planSignature(planNotifications(input()), 'v');
  const c = planSignature(planNotifications(input({ reminderMinutes: 5 })), 'v');
  const d = planSignature(planNotifications(input()), 'no-vibrate');
  expect(a).toBe(b);
  expect(a).not.toBe(c);
  expect(a).not.toBe(d);
});

it('полярна нощ (Тромсø) – всички известия са валидни дати', () => {
  const tromso = locationFromCoords(69.65, 18.96, { names: { bg: 'Тромсø', en: 'Tromsø' } });
  const plan = planNotifications(input({ now: new Date(2026, 11, 20, 8), options: { ...options, location: tromso } }));
  expect(plan.length).toBeGreaterThan(20);
  expect(plan.every((n) => !isNaN(n.at.getTime()))).toBe(true);
});
});
```

Трябва да видиш: Tests: 112 passed, 112 total

npm test

Стъпка 10

Проверка и край на етапа

Етап 4 е готов, когато на телефона:

- При първо отваряне след новия build се появява въпрос за известия. След „Разреши“ в „Настройки → Известия“ пише „Разрешение: Дадено“ и „Планирани N известия – до ...“.
- **Android 14+:** ако се вижда „Точни известия: Изключени“ – натисни „Разреши „Аларми и напомняния““, включи го и се върни. Редът изчезва.
- „Проверка на известията“ → „Известие“ и „Аларма“ → заключи телефона → след 10 сек. идва известие със съответния звук.
- Включи камбанка на следващата молитва и изчакай: известието идва в минутата на молитвата, с правилния текст.
- Напомняне 10 мин. → идва 10 мин. преди молитвата.
- Изключи вибрацията → следващото известие е без вибрация (Android).
- Рестартирай телефона → известията продължават да идват.
- 3 дни подред – без пропуснато известие.
- **Huawei без Google услуги** (ако имаш достъп): известията идват. GPS там още не работи – избери място ръчно; оправяме го в етап 5.

Ако известие закъснее на Xiaomi, Samsung или Huawei: провери „Аларми и напомняния“ и изключи оптимизацията на батерията за Езан. Екранът с инструкции за всеки производител е в етап 5.

Терминал

```
git add .  
git commit -m "Етап 4: известия"  
git tag stage-4
```

Kotlin модульът е написан по шаблона на Expo, но не е компилиран тук (няма Android SDK) – ако build-ът даде грешка в него, изпрати ми я.